IILM UNIVERSITY

MAJOR PROJECT · PHASE-I · IMPLEMENTATION ROADMAP

Detailed Weekly Implementation Roadmap

Intelligent Farm Resource Optimization Using ML and Predictive Analytics

Project title	Intelligent Farm Resource Optimization Using ML and Predictive Analytics
Project code	26P_3CSE139 (as printed on the Phase-I report)
Project type	Major Project — Phase I / Implementation Roadmap
Programme	Bachelor of Technology
Branch	Computer Science and Engineering
Institution	IILM University, Greater Noida
Academic session	2026–27
Project guide	Dr. Swati Vashisht

Project team (names, sections and roll numbers exactly as printed in the Phase-I report)

Name	Roll No.	Section (as in report)
Sachin Raj	2410030656	3CSE21
Ravi Raj Kumar	2410030680	3CSE21
Kartik Sharma	2410030213	3CSE2
Prashant Kumar Yadav	2410030236	3CSE2

Document status

Version 1.0 · prepared 29 October 2026 · planning document for class presentations, faculty review and team coordination.

Nothing in this document has been implemented, trained or tested yet. Every result cell is a placeholder ("To be measured") until the team produces real results.

Table of Contents

About this Document and Source Legend . . . 4

Items in the Phase-I report that need confirmation . . . 4

1 Executive Summary . . . 5

2 Problem Statement, Objectives and Scope . . . 6

2.1 Problem statement [R] . . . 6

2.2 Research gap [R] . . . 6

2.3 Objectives stated in the Phase-I report [R] . . . 6

2.4 Scope . . . 6

2.5 How the components relate . . . 7

3 System Architecture . . . 8

3.1 Design principles . . . 8

3.2 Agricultural background: the soil-water balance . . . 9

3.3 Data provenance classes . . . 9

4 Detailed Dataset Acquisition Plan . . . 11

4.1 Category A: soil-moisture and weather time series . . . 11

4.2 Category B: crop-recommendation datasets . . . 12

4.3 Category C: crop-yield datasets . . . 13

4.4 Category D: soil-nutrient data . . . 14

4.5 Category E: dataset validation . . . 14

5 Twelve-Week Detailed Implementation Roadmap . . . 16

Week 1: Problem Analysis and System Planning . . . 18

Week 2: Dataset Research and Collection . . . 20

Week 3: Data Preprocessing and Exploratory Data Analysis . . . 22

Week 4: Feature Engineering and ML Pipeline . . . 24

Week 5: Soil-Moisture Forecasting . . . 26

Week 6: Irrigation Recommendation Engine . . . 28

Week 7: Crop Suitability and Fertilizer Guidance . . . 30

Week 8: Crop-Yield Prediction . . . 32

Week 9: Resource Optimization . . . 34

Week 10: Dashboard Development . . . 36

Week 11: Testing and Model Evaluation . . . 38

Week 12: Final Integration and Presentation . . . 40

6 Technical Implementation Guide . . . 42

6.1 Recommended folder structure . . . 42

6.2 Illustrative snippets . . . 43

7 Machine Learning Methodology . . . 46

7.1 Soil-moisture regression . . . 46

7.2 Crop classification . . . 46

7.3 Crop-yield regression . . . 46

7.4 Rule-based irrigation recommendation . . . 47

7.5 Scenario-based resource comparison . . . 47

7.6 Why time series need chronological evaluation . . . 47

7.7 Evaluation formulas . . . 48

8 Data Tables and Example Results . . . 49

9 Dashboard Design . . . 50

10 Testing and Evaluation Plan . . . 52

10.1 Model assessment with held-out data . . . 52

10.2 Three levels of evaluation . . . 53

11 Team Responsibilities . . . 54

12 Risk Assessment . . . 55

13 Presentation Preparation . . . 56

14 Viva Preparation . . . 59

15 References and Research Integrity . . . 61

Phase-I report references (as listed in the report) [R] . . . 62

Separating kinds of information . . . 63

16 Final Implementation and Presentation Checklist . . . 64

Implementation . . . 64

Presentation and viva . . . 64

About this Document and Source Legend

This roadmap is built from the team's Phase-I report (the authoritative source for project identity, objectives, scope and research gap) and extended with implementation recommendations. To keep research integrity clear, every substantive statement can be traced to one of five kinds of source, marked with the tags below.

Table 1. Source legend used throughout the document

Tag	Meaning	How to treat it
[R] Report	Taken from the uploaded Phase-I report.	Authoritative for the project. Quote or cite as "Phase-I report".
[A] Additional	Implementation recommendation proposed in this roadmap.	A suggestion. The team and guide may change it.
[V] Verified	External technical fact checked against a public source on 28 October 2026 (see References).	Re-check before submission: APIs and dataset pages change.
[I] Illustrative	Example, placeholder value or simulated scenario.	Never present as a measured or real-world result.
[E] Experimental	Actual result produced by the team's own code.	None exist yet. Fill in only from real runs.

Items in the Phase-I report that need confirmation

The instructions for this document say to preserve the report's wording and flag unclear details rather than invent replacements. The following were noticed while reading the report.

Table 2. Report details preserved as written and flagged for confirmation

#	Observation in the report	Handling in this roadmap
1	Section label is "3CSE21" for Sachin Raj and Ravi Raj Kumar but "3CSE2" for Kartik Sharma and Prashant Kumar Yadav.	Reproduced exactly. Please confirm whether "3CSE2" is a truncation of "3CSE21" or a different section.
2	The cover says "School of Computer Science and Engineering"; the declaration page says "School of Computing Science and Engineering".	Cover wording used. Confirm the official school name.
3	The guide's designation and signature block in the declaration are blank.	Only the name (Dr. Swati Vashisht) is used.
4	Report headings are numbered inconsistently ("References" and "Plagiarism Report" are both numbered 5; the contents page lists sections without page numbers).	Section names used, numbering ignored.
5	The report's six references are listed by title only (no authors, years or DOIs).	Reproduced by title in the References section with a note to complete them before final submission.
6	The literature survey states a water reduction "of up to 40%" for scheduled irrigation in existing approaches.	Treated as a literature claim from the report. It is not used as a target or promise for this project.
7	The report's objectives and scope assume on-farm soil sensors and crop imagery; this roadmap must work without hardware.	Handled explicitly in Section 2 (adaptation table). The original objectives are kept; only the data source changes.

1 Executive Summary

The problem. Many small and medium farms water, fertilise and manage crops by fixed schedules, visual estimates and regional rules of thumb instead of data about soil, crop and weather conditions. The Phase-I report identifies three consequences: inefficient water use, over-application of fertiliser and pesticide, and reactive rather than preventive crop management [R].

Why machine learning helps. Soil moisture, rainfall and temperature change continuously and interact in non-linear ways. Models trained on historical records can forecast the near-term state of the soil, estimate which crops suit given conditions and relate seasonal weather to yield. A transparent recommendation layer then converts these forecasts into actions such as "irrigate", "defer because rain is expected" or "no irrigation needed", each with a stated reason.

Why no physical sensors are required for the prototype. The team has no soil-moisture sensors, IoT devices or experimental farm. The Phase-I feasibility study already notes that public agricultural and climate datasets can address the cold-start problem before pilot sensors exist [R]. This roadmap therefore builds the whole prototype on public datasets, the free Open-Meteo weather APIs, optional NASA SMAP satellite soil-moisture products, user-entered soil-test values, and clearly labelled simulations. Hardware is never compulsory; it is described only as a future extension.

Data and models. Hourly reanalysis data (including modelled soil moisture) are retrieved for a chosen pilot location, aggregated to daily values and turned into lag, rolling-rainfall and seasonal features. Three supervised models are planned: (1) a soil-moisture regression model (persistence baseline, Random Forest, optional gradient boosting); (2) a crop-suitability classifier trained on a public labelled dataset; and (3) a crop-yield regression model trained on public district-level yield statistics. Fertiliser guidance is rule-based and documented, because public data rarely contain dose labels.

From predictions to recommendations. A rule-based engine combines the moisture forecast, the rain outlook and crop-specific thresholds. A simulation compares a fixed-schedule irrigation baseline with a prediction-informed policy in terms of estimated water use, irrigation events and stress days. These are scenario results under stated assumptions, not field-measured savings.

Dashboard and evaluation. A modular Streamlit application with eight pages presents data, forecasts, recommendations, scenario comparisons and model evaluation, and works offline from local datasets. Evaluation uses chronological hold-out testing (MAE, RMSE, R^2) for regressions, stratified testing (accuracy, precision, recall, F1) for classification, and a pytest suite for the application logic.

Expected academic deliverables. Source code, a dataset register and data-quality report, three trained and documented models, the recommendation and scenario modules, the dashboard, a test report, a final report, a presentation and viva preparation material. Results will be reported as measured, including cases where a model fails to beat its baseline.

> **Honesty statement**
> No part of the proposed system has been implemented or tested at the time of writing. All metrics, savings and yield figures in later sections are placeholders or clearly marked illustrations.

2 Problem Statement, Objectives and Scope

2.1 Problem statement [R]

Agriculture must meet the demands of a growing population while water tables fall, fertiliser and energy costs rise and manual labour for crop monitoring declines. Most small and medium farming operations still irrigate, fertilise and apply pesticide on schedules, visual estimates or regional rules of thumb rather than real-time information about soil, crops and weather. The report names three resulting problems: inefficient water usage (calendar-based rather than moisture-, evapotranspiration- or forecast-based irrigation), environmental pollution from uniformly applied fertiliser and pesticide, and reactive management after stress or pest damage has already occurred.

2.2 Research gap [R]

The report observes that many existing models address a single component (yield prediction, pest detection or irrigation) and that the few multi-component systems are either too expensive for farmers or are view-only dashboards. Farmers need a *unified optimization plan* that considers water, fertiliser and labour together, built on a predictive pipeline that integrates soil, weather, imagery and historical yield data. It also notes that yield prediction is mostly accurate at regional level but not at farm or field level because location-specific data are scarce.

2.3 Objectives stated in the Phase-I report [R]

Table 3. Original objectives and how this roadmap addresses them

Obj.	Wording in the report (condensed)	Realisation without hardware (roadmap adaptation [A])	Weeks
1	Create a data pipeline that consolidates and cleans multi-source data per field zone: soil sensors (moisture, pH, NPK), weather stations/APIs, crop imagery.	Pipeline over Kaggle/ICRISAT files, Open-Meteo reanalysis and forecast APIs, optional SMAP, and user-entered soil-test values. "Zone" becomes a pilot location or district. Imagery is deferred.	2–4
2	Build ML models predicting short-term irrigation need, nutrient need per zone and potential yield per zone.	Soil-moisture forecast feeds an irrigation rule engine; crop classifier plus rule-based nutrient guidance; yield regression at district-season level.	5–8
3	Create a recommender that turns predictions into an optimization-based plan (for example "irrigate zone three for 20 minutes tomorrow").	Rule-based recommendation engine with explanations. Irrigation *duration* is produced only if the user supplies system parameters (flow rate, area, efficiency).	6, 9
4	Evaluate resource use (water, fertiliser) and yield-prediction accuracy against a fixed-interval control.	Simulated comparison of fixed-schedule versus prediction-informed irrigation; yield accuracy on held-out years. No field savings are claimed.	9, 11
5	Develop an application that needs minimal technical knowledge to use.	Streamlit dashboard with eight pages, plain-language explanations and offline mode.	10

2.4 Scope

In scope in the report [R]: collection of soil moisture, temperature, humidity, rainfall and basic soil-nutrient data for a defined pilot area; ML models for irrigation scheduling, fertiliser recommendation and yield prediction using historical and/or public agricultural data plus pilot sensor data; automatic action recommendations; an app or dashboard for current and forecast conditions; and evaluation of the irrigation recommendation against a fixed-schedule baseline.

Adaptation for this roadmap [A]. The report's feasibility study treats low-cost sensors as optional supplements and public data as sufficient to avoid the cold-start problem [R]. The roadmap uses that route and makes the following explicit.

Table 4. Report scope items versus implementation status in this roadmap

Item from report scope or abstract	Status in this roadmap	Reason
Soil-moisture, temperature, humidity, rainfall data	Included (modelled and forecast values from public sources)	No sensors available.
Basic soil-nutrient data	Included via public crop datasets and user-entered soil-test values	Weather data contain no nutrient information.
Irrigation scheduling, fertiliser recommendation, yield prediction	Included	Core of the project.
Recommendation system and dashboard	Included	Core of the project.
Comparison with fixed-schedule irrigation	Included as simulation	No field trial possible.
Crop imagery, NDVI and CNN-based health monitoring; pest detection	Deferred (future work)	Needs imagery and labels outside the 12-week plan.
Targeted pesticide application; labour optimization	Deferred (future work)	No data; outside the models proposed for implementation.
IoT sensor nodes, edge computing unit	Optional future extension	Hardware is not compulsory.

2.5 How the components relate

The seven themes of the project form one chain. Weather data are the shared input. The soil-moisture forecast tells the irrigation engine what the soil will look like tomorrow; the rain outlook decides whether watering can wait. Crop suitability and nutrient guidance describe what to grow and what the soil may lack. Yield prediction links seasonal weather and crop choice to expected output. Resource optimization compares policies for using water. The dashboard presents everything to a non-technical user.

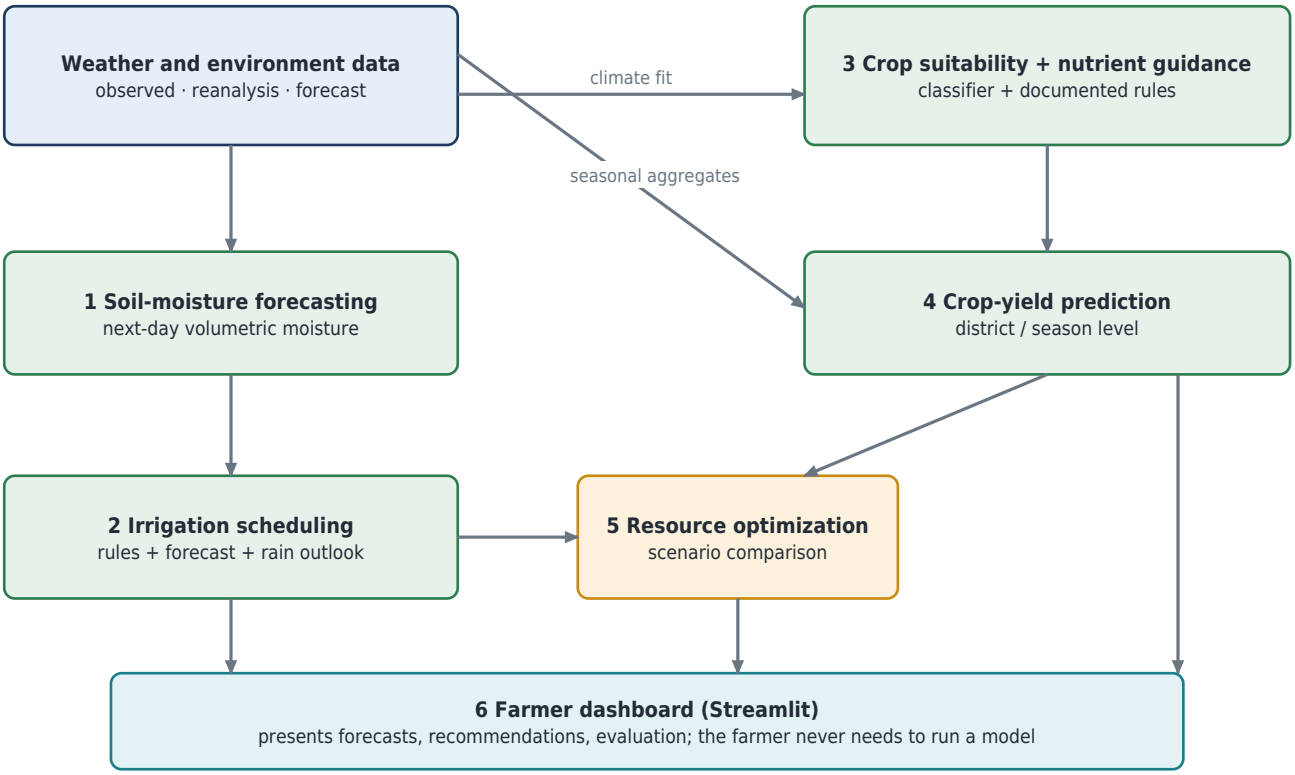


Figure 1. *Relationships between the project components. Arrows show the direction in which outputs are used.*

Original objectives versus additional recommendations

Original objectives (Section 2.3) come from the Phase-I report [R]. Everything else added by this roadmap (Open-Meteo as data source, persistence baseline, chronological splits, 12-week structure, eight dashboard pages, testing matrix, team matrix) is an implementation recommendation [A] that supports those objectives.

3 System Architecture

The architecture follows the required flow: data sources, collection, validation and preprocessing, feature engineering, machine-learning models, recommendation engine, resource optimization, Streamlit dashboard, and evaluation and reporting [A]. Data sources are optional. Every stage reads from and writes to a local store, so the system runs from local datasets when the internet is unavailable.

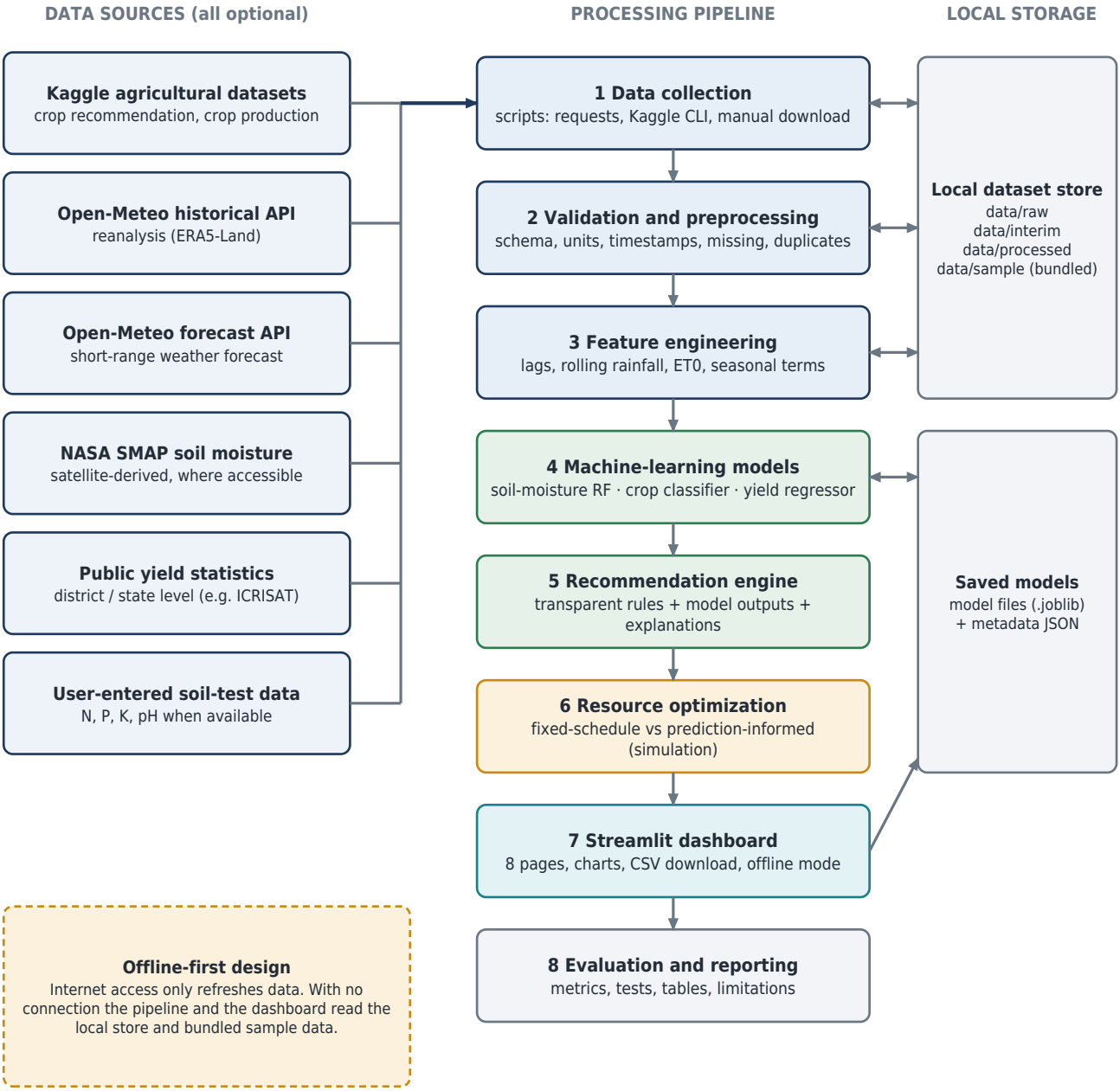


Figure 2. *Proposed system architecture (design, not an implemented system). Left: optional data sources; centre: pipeline; right: local storage that enables offline operation.*

3.1 Design principles

- **Optional sources, local truth.** Online downloads are cached as CSV files in `data/raw`. Later stages read only the cache. With no connection the app uses cached or bundled sample data and shows a clear "offline" label.
- **Separation of concerns.** One module per responsibility (collection, preprocessing, features, models, rules, optimization, evaluation, UI) so that four people can work in parallel and tests can target each part.

- **Transparency over accuracy theatre.** Recommendations are rule-based and explained. Models supply numbers (forecast moisture, probabilities, yield estimates); rules supply decisions.
- **Provenance everywhere.** Every column carries a provenance class (Table below). The dashboard shows a badge so a modelled value is never mistaken for a sensor reading.
- **Reproducibility.** Fixed random seeds, pinned package versions, saved model metadata (feature list, training dates, library versions).

3.2 Agricultural background: the soil-water balance

Irrigation decisions rest on a simple idea: the water stored in the root zone rises with rainfall and irrigation and falls with evapotranspiration (crop and soil water use) and drainage. ERA5-Land reports modelled volumetric soil water in four layers (0–7, 7–28, 28–100 and 100–255 cm) [V]. A root zone of a few tens of centimetres therefore spans the first two or three layers, which is why the roadmap uses a depth-weighted value instead of the 0–7 cm layer alone.

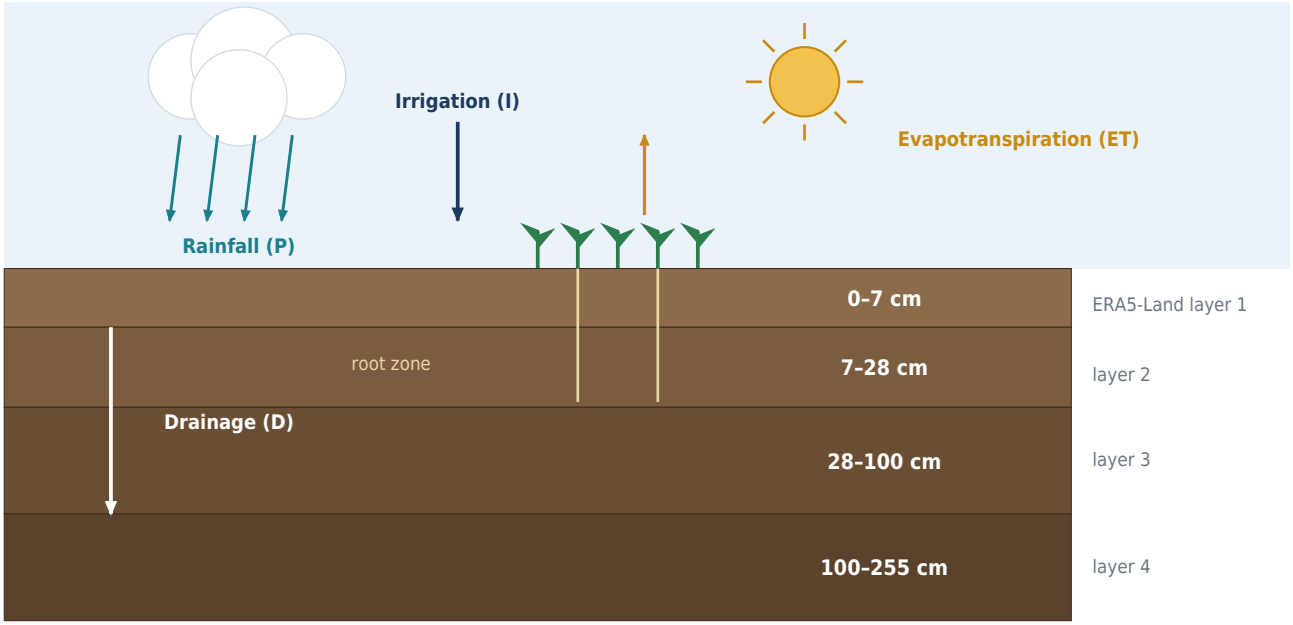


Figure 3. *Conceptual soil-water balance (schematic, not to scale). Layer depths follow the ERA5-Land layers; arrows show water inputs and losses.*

3.3 Data provenance classes

The roadmap must distinguish carefully between kinds of data. Table below defines six classes. Only the first is a direct physical measurement; none of them will be available from our own sensors.

Table 5. Data provenance classes and how each is used in this project

Class	Meaning	Examples here	Caution
Measured	Recorded by a physical instrument at the site or a weather station.	Not available from the team's own hardware. Published station data in Kaggle files may belong here (check each file).	Check calibration, units and location.
Satellite-derived	Estimated from remote-sensing signals.	NASA SMAP surface soil moisture (SPL3SMP, 36 km grid) [V]	Coarse footprint, surface layer only, gaps from retrieval conditions.
Reanalysis estimate	Gridded model state constrained by assimilated observations.	ERA5 / ERA5-Land served by the Open-Meteo Historical API (ERA5-Land about 0.1°, hourly, from 1950) [V]	Modelled soil moisture is **not** a sensor reading; grid cell is about 11 km.
Forecast	Output of a numerical weather prediction model for future time.	Open-Meteo Forecast API	Has forecast error; differs from the reanalysis used for training.
User-entered	Values typed by the user.	Soil-test N, P, K, pH; crop; soil parameters; irrigation system parameters	Validate ranges and units; cannot be verified by the app.
Simulated	Generated by a model or rule written by the team.	Scenario water balance; rule-generated labels	Never present as real-world outcomes.

4 Detailed Dataset Acquisition Plan

This section explains how the team can obtain suitable data without any sensors. **No dataset has been downloaded for this document.** Descriptions below come from public documentation and dataset pages checked on 28 October 2026 [V]; anything not verifiable from documentation is marked "to be checked". Before using any dataset the team must open its page, read the description, check the licence and record both in the dataset register.

Table 6. Dataset register (initial plan; status updated by the data owner)

ID	Dataset / service	Role	Provenance class	Licence / terms (to confirm on page)	Status
DS-01	Open-Meteo Historical Weather API (ERA5-Land / ERA5)	Moisture and weather time series	Reanalysis	CC BY 4.0; free for non-commercial use, no key; attribution required [V]	Planned
DS-02	Open-Meteo Forecast API	Rain/temperature outlook for recommendations	Forecast	Same terms as DS-01 [V]	Planned
DS-03	Kaggle: Crop Recommendation Dataset (atharvaingle)	Crop classification	Compiled/augmented (check page)	Per Kaggle dataset page; do not assume	Planned
DS-04	Kaggle: Crop Production in India (abhinand05)	District yield (area, production)	Government statistics compiled by uploader	Per Kaggle dataset page	Planned
DS-05	ICRISAT District Level Database (DLD)	Alternative/verified yield statistics	Compiled official statistics	Open access with citation; data provided "as is" [V]	Planned
DS-06	NASA SMAP L3 36 km soil moisture (SPL3SMP)	Optional satellite check of moisture	Satellite-derived	Openly shared; Earthdata Login required [V]	Optional
DS-07	User-entered soil-test values (dashboard form)	Fertiliser guidance inputs	User-entered	Entered by user	To build
DS-08	Kaggle fertiliser-name datasets (candidates)	Optional fertiliser-type classification	Check each	Check each	Search only
DS-09	Scenario parameters (config file)	Resource simulation	Simulated	Team-authored	To build

4.1 Category A: soil-moisture and weather time series

A forecasting model needs **timestamped** soil moisture together with weather. Without sensors, the practical choices are reanalysis (Open-Meteo, ERA5-Land), satellite products (SMAP) or a public sensor dataset found on Kaggle.

Table 7. A1. Open-Meteo Historical Weather API (primary source for the moisture model)

Purpose	Daily/hourly series of modelled soil moisture plus rainfall, temperature, humidity and evapotranspiration for the pilot location. Trains the soil-moisture model and drives the irrigation engine.
Source and URL	Open-Meteo Historical Weather API. Docs: https://open-meteo.com/en/docs/historical-weather-api. Endpoint: `https://archive-api.open-meteo.com/v1/archive` [V]
Required columns	time; soil_moisture_0_to_7cm, soil_moisture_7_to_28cm, soil_moisture_28_to_100cm (m³/m³); precipitation (mm); temperature_2m (°C); relative_humidity_2m (%); et0_fao_evapotranspiration (mm). Confirm exact variable names on the docs page when coding; names are occasionally extended.
Target variable	Next-day mean soil moisture of the chosen layer(s).
File format	JSON by default; CSV and XLSX available. The pipeline saves a CSV copy in data/raw/openmeteo/.
Granularity	ERA5-Land: 0.1° (about 11 km), hourly, 1950 to present, about 5 days behind real time. ERA5: 0.25° (about 25 km) [V]. Soil moisture layers: 0–7, 7–28, 28–100, 100–255 cm [V]
Download	No key for non-commercial use. Call the endpoint with latitude, longitude, start_date, end_date, hourly=..., timezone=Asia/Kolkata using requests; save the response; record the request URL in the register.
Cleaning	Parse timestamps; aggregate hourly to daily (mean for moisture, temperature and humidity; sum for precipitation and ET0); check range 0–1 m³/m³ for moisture; check non-negative rainfall.
Intended model	Soil-moisture regression (persistence baseline, Random Forest, optional gradient boosting).
Limitations	Modelled values, not field measurements. Grid-cell average (about 11 km). Cannot represent a specific field's soil texture, irrigation or drainage. Results describe skill at predicting a model's soil-moisture state.
Licence	Data under CC BY 4.0; free API for non-commercial use without a key. Acknowledge "Weather data by Open-Meteo.com" in the app and report [V]. Re-read the terms at https://open-meteo.com/en/features#terms.

Table 8. A2. NASA SMAP L3 radiometer soil moisture (optional satellite cross-check)

Purpose	Independent satellite estimate of surface soil moisture to compare qualitatively with the reanalysis series. Optional: skip if time is short.
Source and URL	NSIDC DAAC product SPL3SMP: https://nsidc.org/data/spl3smp/ [V]
Required columns	Soil-moisture field, retrieval quality flags, latitude/longitude grid (HDF5 files).
File format / granularity	HDF5 granules; daily composite on a 36 km EASE-Grid 2.0 [V].
Download	Free NASA Earthdata Login account required [V]. Use Earthdata Search to subset by area and date, then download the files.
Cleaning	Extract the pixel covering the pilot site; mask flagged retrievals; keep missing days missing.
Limitations	Surface layer only; 36 km footprint is far larger than a farm; gaps; different physical quantity from the multi-layer reanalysis.
Licence / citation	Openly shared; cite DOI https://doi.org/10.5067/4XXOGX0OOW1S (Version 9) [V]

Kaggle soil-moisture sensor datasets

Kaggle hosts various IoT soil-moisture and irrigation datasets. Their quality and provenance differ widely, and some are small or synthetic. Search kaggle.com/datasets for "soil moisture", then read the description, check timestamps and licence, and record the decision in the register. Do not use a dataset whose origin cannot be established as evidence of real-world performance.

Modelled is not measured

Reanalysis soil moisture is produced by a land-surface model driven by atmospheric data. It is valuable for development and teaching, but it must always be labelled "modelled" and never described as a sensor measurement.

4.2 Category B: crop-recommendation datasets

Table 9. B. Crop recommendation (classification)

Purpose	Train a classifier that suggests suitable crops from soil nutrient and climate inputs.
Source and URL	Kaggle: Crop Recommendation Dataset by Atharva Ingle: https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset **[V]** (address as cited in published papers; open the page to confirm current details).
Required columns	N, P, K, temperature, humidity, ph, rainfall; label (crop).
Target variable	label (crop name).
File format / size	CSV (Crop_recommendation.csv). Described in the literature as 2,200 rows, 22 crops, 100 samples per crop, seven predictors, no missing values **[V]**.
Download	Kaggle website (login) or CLI: `kaggle datasets download -d atharvaingle/crop-recommendation-dataset --unzip -p data/raw/crop_reco`. Follow Kaggle's current instructions for credentials.
Cleaning	Verify columns and ranges; check duplicates; keep the label encoded with LabelEncoder or as strings; no scaling needed for tree models (scale for KNN/logistic regression).
Intended model	Crop classifier (Random Forest, with Decision Tree, KNN, majority-class baselines).
Limitations	Not a moisture dataset: it has no time dimension, so it cannot train the moisture forecaster. The set is small, perfectly balanced and has no location or season columns; its origin is not field-level measurement (read the dataset page for provenance). High accuracy on a random split may not transfer to real fields.
Licence	See the Kaggle dataset page; do not assume redistribution rights. Cite the dataset and uploader.

Crop-recommendation data are appropriate for **crop classification** only. N, P and K in this dataset are in the dataset's own units; they are not interchangeable with kg/ha values on a laboratory soil-test report.

4.3 Category C: crop-yield datasets

Table 10. C1. Kaggle: Crop Production in India (district level, quick start)

Purpose	Learn the relationship between crop, season, year, region, weather and yield.
Source and URL	https://www.kaggle.com/datasets/abhinand05/crop-production-in-india **[V]** (address as cited in public analyses).
Columns	State_Name, District_Name, Crop_Year, Season, Crop, Area, Production. Reported coverage 1997 to 2015 **[V]**.
Target variable	Yield = Production / Area (derived, in tonne per hectare when production is in tonnes and area in hectares). The file has no ready-made yield column.
Granularity	District-level annual figures by crop and season.
Download	`kaggle datasets download -d abhinand05/crop-production-in-india --unzip -p data/raw/crop_prod`
Cleaning	Drop rows with missing production, zero or missing area; check units per crop (some crops are reported in non-tonne units in some versions, to be checked); standardise names (state, district, crop spelling); remove duplicates; examine outliers.
Intended model	Yield regression with baselines (crop mean, previous-year yield).
Limitations	Data end in 2015; district-level only; no farm management or soil data; crop statistics have reporting inconsistencies.
Licence	See Kaggle page; underlying figures originate from government statistics. Cite both.

Table 11. C2. ICRISAT District Level Database (more authoritative alternative)

Purpose	Use official-statistics-based area, production and yield for major crops as a cross-check or replacement for the Kaggle file.
Source and URL	ICRISAT DLD portal: http://data.icrisat.org/dld/index.html; crop files: http://data.icrisat.org/dld/src/crops.html **[V]**
Content	District-wise yearly area, production and yield for about 20 major crops; the database covers many states and years (several decades from 1966) with documented district-boundary conventions **[V]**.
Format	Downloadable data files (CSV/Excel). Check the file layout before coding.
Limitations	Boundary changes over time; some states do not report some series; data are provided "as is" **[V]**.
Licence / citation	Open access; cite as requested (DOI https://doi.org/10.21421/D2/XFB1BZ) **[V]**

Table 12. Yield at different spatial levels and what each can support

Level	Meaning	What a model can claim
Farm / field	Yield of one field (needs farm records or harvest sensors).	Management advice for one field. Not available in public data.
District	Total production divided by area across a district.	Regional trends and weather sensitivity. Our feasible level.
State	Aggregate over many districts.	Broad policy-scale trends; hides local variation.

The Phase-I report itself notes that yield prediction is mostly accurate regionally but not at farm level because location-specific data are scarce **[R]**. The yield model will therefore be described as a **district-level** model and not as a prediction for a particular farm.

4.4 Category D: soil-nutrient data

Nutrient guidance needs measured soil-test values: nitrogen (N), phosphorus (P), potassium (K), pH and ideally organic carbon, electrical conductivity and micronutrients, each with a unit and a test method. **Weather datasets do not contain these measurements.** The public options are: (1) the N, P, K and pH columns of the crop-recommendation dataset, which are suitable for crop classification; (2) values entered by the user from a laboratory soil-test report (for example, a Soil Health Card issued under the Government of India scheme, or a university laboratory report); and (3) small Kaggle fertiliser datasets that predict a fertiliser *name* but not a *dose*.

Table 13. D. Soil-nutrient inputs and their use

Input	Typical unit on a lab report	Source in this project	Use
N, P, K availability	kg/ha or ppm, with a rating (low/medium/high)	User form on the Fertilizer page	Rule-based guidance
pH	pH units	User form; crop dataset	Rule-based guidance; classifier feature
Organic carbon, EC, micronutrients	% / dS/m / ppm	User form (optional)	Display and notes only
N, P, K in crop dataset	Dataset's own scale	DS-03	Classifier feature only. Do not convert to kg/ha

Why fertiliser dose is not predicted

A dose depends on crop, soil test, yield target and local agronomic recommendations. Public datasets rarely contain a supervised dose label, so a model trained on them would learn invented patterns. Week 7 therefore uses documented, source-cited rules rather than a dose regression.

4.5 Category E: dataset validation

Every dataset passes the same checklist before it is used. The checks are automated in `preprocessing.py` and summarised in the data-quality report (Week 3).

- **Files and columns:** expected file names, encoding, delimiter; expected columns present with expected dtypes.

- **Missing values:** count and percentage per column; pattern (random, seasonal, block).
- **Units:** fraction vs percent for moisture; mm vs m for rainfall; °C vs K; tonne vs quintal; hectare vs acre.
- **Dates:** parseable, timezone-consistent, sorted, regular frequency, no impossible dates, no duplicate timestamps.
- **Duplicates:** exact duplicate rows; duplicate keys (for example district, crop, year, season).
- **Provenance:** who produced the data, when, how, which licence; stored in the register.
- **Target suitability:** is the target a real measurement, a modelled value, or a rule-generated label? Does it vary enough? Is it independent of the features?

Table 14. Data dictionary template (copy one row per column)

Column	Description	Unit	Type	Allowed range	Provenance class	Missing code	Notes
(name)	(meaning)	(unit)	(float/int/str/datetime)	(min–max)	(class)	(NaN/-999)	(remarks)
sm_0_7cm [I]	Modelled soil moisture, 0–7 cm layer, daily mean	m³/m³	float	0–1	Reanalysis	NaN	Example entry
rain_mm [I]	Daily precipitation total	mm	float	≥ 0	Reanalysis	NaN	Example entry

Table 15. Sample schema of the daily modelling table [I] (column names suggested; no values shown because no data have been downloaded)

Column	Type	Description	Role
date	datetime (index)	Local calendar day	Index
sm_0_7cm, sm_7_28cm, sm_28_100cm	float	Daily mean modelled soil moisture (m³/m³)	Input and basis of target
rain_mm	float	Daily precipitation total (mm)	Input
tmean_c, tmax_c, tmin_c	float	Daily temperature statistics (°C)	Input
rh_mean	float	Daily mean relative humidity (%)	Input
et0_mm	float	Reference evapotranspiration (mm/day)	Input
*_lag1 ... *_lag7, rain_sum3d/7d/14d, doy_sin, doy_cos	float	Engineered features (Week 4)	Input
target_sm_next	float	Moisture of the chosen layer at day t+1	Target

5 Twelve-Week Detailed Implementation Roadmap

The plan below is a suggestion [A] built around the twelve weeks requested. Week 1 starts on whichever date the team chooses. Each week follows the same fifteen-part structure so it can be used for planning, review meetings and class presentations: (1) objective, (2–3) technical background, (4) activities, (5) steps, (6) tools, (7) data, (8) files, (9) expected output, (10) deliverables, (11) tests, (12) common problems, (13) class explanation, (14) slides, (15) checklist.

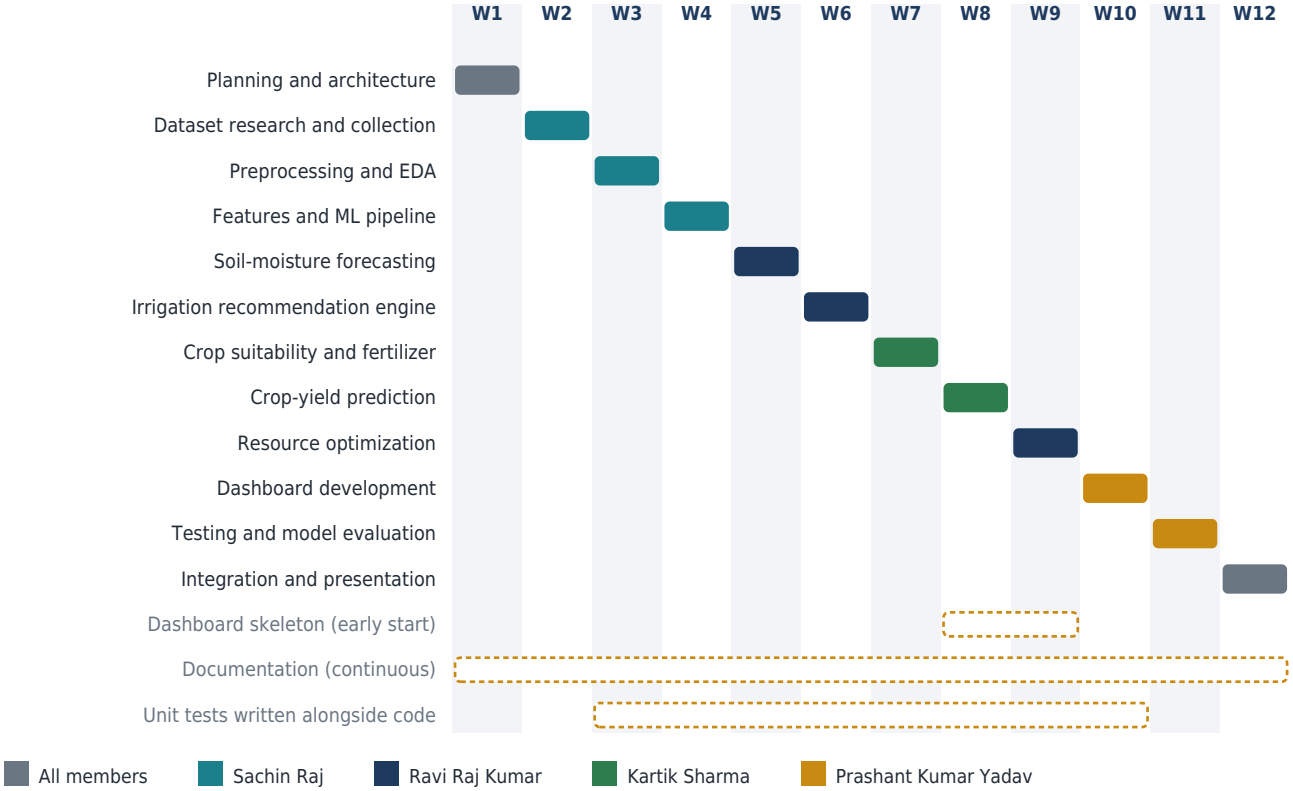


Figure 4. *Suggested twelve-week timeline by workstream. Colours indicate the proposed lead; dashed bars are activities that run alongside the main plan. Dates are not fixed.*

Table 16. Weekly timeline summary

Week	Title	Main output	Suggested lead
1	Problem Analysis and System Planning	Scope, architecture, repository	All members (planning)
2	Dataset Research and Collection	Raw data and dataset register	Sachin Raj
3	Data Preprocessing and Exploratory Data Analysis	Clean daily tables, data-quality report	Sachin Raj
4	Feature Engineering and ML Pipeline	Features, targets, leak-free splits	Sachin Raj
5	Soil-Moisture Forecasting	Soil-moisture model and baseline comparison	Ravi Raj Kumar
6	Irrigation Recommendation Engine	Irrigation recommendation engine	Ravi Raj Kumar
7	Crop Suitability and Fertilizer Guidance	Crop classifier and nutrient guidance	Kartik Sharma
8	Crop-Yield Prediction	District yield model	Kartik Sharma
9	Resource Optimization	Simulated resource comparison	Ravi Raj Kumar
10	Dashboard Development	Streamlit dashboard	Prashant Kumar Yadav
11	Testing and Model Evaluation	Test suite and frozen evaluation	Prashant Kumar Yadav
12	Final Integration and Presentation	Release, report, slides, viva	All members

Week 1: Problem Analysis and System Planning

1. Objective	Agree a precise, achievable problem statement and scope for a sensor-free prototype; finalise the architecture, priorities and responsibilities.
Suggested lead	All members (planning); repository set-up by Prashant Kumar Yadav **[A]**
6. Software and tools	Git and GitHub, VS Code, Python 3.10 or newer, venv, draw.io or Mermaid, a shared task board (Trello, Notion or GitHub Projects).
7. Dataset requirements	None downloaded this week. Shortlist candidate sources (Section 4).
8. Python files / modules	README.md, requirements.txt, config.yaml, docs/problem_statement.md, docs/traceability.csv, docs/constraints.md.
9. Expected output	Approved scope and priorities; architecture diagram v1; working repository skeleton on every machine.
10. Deliverables	Scope and priority sheet (MoSCoW); Traceability table; Architecture diagram v1; Responsibility matrix; Repository link and meeting notes.

2–3. Technical background

The Phase-I report proposes an integrated precision-agriculture system with sensing, ML models, a recommender and a dashboard, and its feasibility study already accepts public datasets for the cold-start phase **[R]**. Week 1 converts that proposal into a hardware-free plan. Each report objective is classified as *implementable with public data* (irrigation need, crop suitability, yield, dashboard), *adapted* (zone becomes location or district; sensor moisture becomes reanalysis moisture) or *deferred* (imagery, pesticide targeting, labour, IoT). A clear scope now prevents the most common project failure: trying to build everything.

4. Detailed activities

- Read the Phase-I report line by line and extract every objective, scope item and constraint.
- Build a traceability table: report objective → module → week → owner.
- Rank features with MoSCoW (must, should, could, will-not) and fix the minimum viable product.
- Choose one pilot location and one or two crops (suggestion: Greater Noida / western Uttar Pradesh, wheat in the rabi season and rice in the kharif season) **[A]**.
- Record constraints: no sensors, free APIs only, offline operation, 12 weeks, laptop-class compute.
- Draft the architecture diagram, repository layout and weekly meeting routine.

5. Step-by-step implementation procedure

1. Print or share the report; each member highlights objective sentences in a different colour.
2. Merge highlights into `docs/traceability.csv` with columns objective_id, report_text, module, week, owner, status.
3. Write the one-paragraph problem statement and five research questions (for example: *Can a Random Forest beat persistence for next-day soil moisture?*).
4. Run a 60-minute MoSCoW session; move imagery, pesticide, labour and IoT to "will not (this phase)".
5. Create `docs/constraints.md` listing data, compute, time and hardware constraints.
6. Draw the architecture (draw.io or Mermaid) and agree the folder structure from Section 6.
7. Create the Git repository, a virtual environment and `requirements.txt`; every member clones and runs `pip install -r requirements.txt`.
8. Fill and sign off the responsibility matrix (Section 11); book a fixed weekly slot with the guide.

11. Testing and verification criteria

- Every report objective appears in the traceability table with an owner and a week.
- A fresh clone installs and imports the stack on every member's machine.
- The guide has seen and approved the scope (record date and comments).
- No planned task needs physical hardware.

12. Common problems and solutions

Table 17. Week 1: common problems and suggested solutions

Problem	Solution
Scope keeps growing	Apply MoSCoW; anything not traceable to a report objective is "could".
Members use different Python versions	Fix a version (3.10 or newer), use venvs, commit `requirements.txt` from `pip freeze`.
Disagreement on the pilot location	Pick one primary and one backup location; the pipeline takes latitude and longitude as parameters, so switching is cheap.

13. Short explanation for the class

Our report proposes an intelligent precision-agriculture system. Because we have no sensors, we are building the same pipeline on public data, and we have marked clearly what is built now, what is adapted and what is future work.

14. Suggested presentation slides and screenshots

- Slide: project title and team
- Slide: problem statement and research gap
- Figure: architecture diagram v1
- Table: report objective → module → week

15. Checklist

- ☐ Report objectives extracted
- ☐ Traceability table complete
- ☐ MoSCoW agreed
- ☐ Pilot location and crops chosen
- ☐ Architecture diagram drawn
- ☐ Repository created and cloned by all
- ☐ Roles agreed and signed off
- ☐ Guide meeting scheduled

Week 2: Dataset Research and Collection

1. Objective	Obtain all planned datasets, keep raw files untouched, and document source, units, licence and limitations in a dataset register.
Suggested lead	Sachin Raj **[A]**
6. Software and tools	Python, requests, pandas, Kaggle CLI, browser for Earthdata Search, hashlib, Git (data folders in .gitignore).
7. Dataset requirements	DS-01, DS-02, DS-03, DS-04, DS-05 (required); DS-06 (optional).
8. Python files / modules	src/farmopt/data_collection.py, data/raw/MANIFEST.json, docs/dataset_register.csv, docs/licences.md.
9. Expected output	All raw files in place, reproducible download script, completed dataset register.
10. Deliverables	Raw data folders with manifest; data_collection.py; Dataset register v1; Licence and attribution notes.

2–3. Technical background

Without sensors, data quality and provenance decide whether the project is credible. The reanalysis soil-moisture series (Open-Meteo, ERA5-Land, about 0.1° hourly) is modelled, not measured **[V]**. Crop-recommendation data have no time dimension; district yield data have no soil or management information. Collecting the data is therefore as much about understanding what each dataset can and cannot support as about downloading files.

4. Detailed activities

- Create a Kaggle account and API credentials; download the two Kaggle datasets.
- Write `data_collection.py` to fetch Open-Meteo historical data for the pilot location, ideally 2015 to the latest available date.
- Download one ICRISAT district file for cross-checking yield statistics.
- Optionally request SMAP data through Earthdata Search for the pilot area.
- Store everything unmodified in `data/raw` and write a metadata JSON per file (source URL, access date, checksum).
- Complete the dataset register with licence and limitation notes.

5. Step-by-step implementation procedure

1. Create folders `data/raw/openmeteo, crop_reco, crop_prod, icrisat, smap`.
2. Install the Kaggle CLI (`pip install kaggle`) and configure credentials as described in Kaggle's current API documentation.
3. Run `kaggle datasets download -d atharvaingle/crop-recommendation-dataset --unzip -p data/raw/crop_reco` and the equivalent for `abhinand05/crop-production-in-india`.
4. Write `fetch_archive_hourly()` (snippet in Section 6) with a timeout, `raise_for_status()` and a CSV cache; run it for the pilot coordinates.
5. Download the ICRISAT crop file from the DLD portal; note the page, date and citation.
6. Compute SHA-256 hashes (`hashlib`) of all raw files and store them in `data/raw/MANIFEST.json`.
7. Fill the register: ID, name, URL, access date, licence, provenance class, units, time span, limitations, owner.
8. Copy one small sample of each file to `data/sample/` for offline demos (check each licence allows this; otherwise keep samples private).

11. Testing and verification criteria

- Re-running the script reproduces identical files (hash match) for fixed date ranges.
- Each register row has source URL, access date, licence and limitations.
- The script fails gracefully (clear message) when offline.
- No raw file has been edited by hand.

12. Common problems and solutions

Table 18. Week 2: common problems and suggested solutions

Problem	Solution
Kaggle download fails	Check credentials and that the dataset page has been opened while logged in; otherwise download manually from the website and record this in the register.
Open-Meteo returns an error or empty values	Check parameter names against the docs page, keep the date range moderate, and verify the latest available date (reanalysis lags real time by days).
Unsure whether a file may be shared	Do not commit it. Keep raw data out of Git and share only the download script.

13. Short explanation for the class

We use public data and label each source: reanalysis is modelled, Kaggle files are compiled datasets, and everything is documented in a register with its licence and limitations.

14. Suggested presentation slides and screenshots

- Table: dataset register
- Figure: data provenance classes
- Screenshot: terminal showing a successful download (take it for real)

15. Checklist

- ☐ Kaggle datasets downloaded
- ☐ Open-Meteo series saved
- ☐ ICRISAT file saved
- ☐ Manifest with hashes
- ☐ Register completed
- ☐ Licences read
- ☐ Sample data prepared for offline use

Week 3: Data Preprocessing and Exploratory Data Analysis

1. Objective	Clean and standardise every dataset, understand its structure through charts, and publish a data-quality report.
Suggested lead	Sachin Raj **[A]**
6. Software and tools	pandas, NumPy, Plotly (or Matplotlib), Jupyter or VS Code notebooks, pytest for validators.
7. Dataset requirements	DS-01, DS-03, DS-04 (and DS-05 for cross-check).
8. Python files / modules	src/farmopt/preprocessing.py, notebooks/01_eda.ipynb, reports/data_quality.md, data/processed/*.csv.
9. Expected output	Clean, documented, validated daily tables and a set of EDA figures.
10. Deliverables	preprocessing.py with validators; Processed datasets; EDA notebook and figures; Data-quality report.

2–3. Technical background

Data cleaning errors silently corrupt models. For time series the rules are stricter: values must be ordered, regularly spaced and consistently timestamped, and gap filling must not use future information. EDA also reveals whether the problem is learnable (is moisture predictable from rainfall?) and which crop labels are separable. Aggregation from hourly to daily should use the right statistic: mean for state variables such as moisture and temperature, sum for fluxes such as rainfall and evapotranspiration.

4. Detailed activities

- Inspect shape, dtypes, ranges, missing values and duplicates in each dataset.
- Standardise column names (snake_case), units, date formats and timezone.
- Aggregate hourly weather and moisture to daily values.
- Handle missing values with documented rules; flag rather than hide them.
- Create exploratory charts and a written data-quality report.

5. Step-by-step implementation procedure

1. Write `validate_schema(df, spec)` that checks required columns, dtypes and ranges and returns a list of problems.
2. Parse timestamps with `pd.to_datetime`, set a sorted `DatetimeIndex`, assert no duplicate timestamps.
3. Convert hourly to daily: `df.resample('D').agg({'sm_0_7cm':'mean','rain_mm':'sum', ...})`.
4. Check units: moisture within 0–1 m³/m³, humidity 0–100 %, rainfall at least 0, temperature within a plausible range for the location.
5. Missing values: for short gaps (up to 2 days) use time interpolation only if the same rule is applied to every split; otherwise leave NaN and drop rows after feature creation.
6. Crop dataset: remove exact duplicates, check class counts, check value ranges per feature. Yield dataset: drop rows with missing production or zero area, derive yield, inspect outliers by crop.
7. Plot: moisture and rainfall time series, monthly boxplots (seasonality), rain vs next-day moisture change, correlation heatmap, crop class counts, yield distribution by crop.
8. Generate `reports/data_quality.md` (counts of rows removed and why) and save processed files to `data/processed`.

11. Testing and verification criteria

- Validators reject a file with a missing column, wrong dtype or out-of-range value (unit tests).
- Processed daily table has one row per day, sorted, no duplicate dates.
- Row counts before and after cleaning are logged and explained.
- Interpolation or filling is never applied across the train/test boundary in a way that uses future data.

12. Common problems and solutions

Table 19. Week 3: common problems and suggested solutions

Problem	Solution
Timestamps shift by hours or a day	Request a fixed timezone from the API and convert once; verify with a known rainfall event.
Hourly data have gaps	Resample and count missing hours per day; mark days with too many gaps as missing.
Yield outliers (extremely high values)	Check units and tiny areas; document any removal rule rather than deleting silently.

13. Short explanation for the class

Before any model, we validate every file: columns, units, dates, duplicates and missing values, and we publish a data-quality report so the faculty can see what was cleaned and why.

14. Suggested presentation slides and screenshots

- Figure: moisture and rainfall time series (from real downloaded data)
- Figure: class distribution of the crop dataset
- Table: data-quality summary

15. Checklist

☐ Schema validators written
☐ Daily table created
☐ Units verified
☐ Missing and duplicate handling documented
☐ EDA charts saved
☐ Data-quality report written
☐ Processed files saved

Week 4: Feature Engineering and ML Pipeline

1. Objective	Create leak-free features and targets, define chronological train/validation/test splits and build a reusable pipeline.
Suggested lead	Sachin Raj with Ravi Raj Kumar **[A]**
6. Software and tools	pandas, NumPy, scikit-learn (Pipeline, ColumnTransformer, StandardScaler where needed), pytest.
7. Dataset requirements	Processed daily table from Week 3; crop and yield tables.
8. Python files / modules	src/farmopt/features.py, scripts/make_dataset.py, tests/test_features.py, config.yaml (split dates, horizon).
9. Expected output	Feature table, target column, documented split dates and passing leakage tests.
10. Deliverables	features.py; Feature and target documentation; Split definition with dates; Leakage test suite.

2–3. Technical background

Soil moisture is strongly autocorrelated, so yesterday's value is already a strong predictor; useful models add information about recent rainfall, evaporative demand and season. Data leakage occurs when the model sees information it would not have at prediction time. The main leakage routes here are shuffled splits, targets shifted in the wrong direction, rolling windows that include future days, preprocessing fitted on all data, and using observed future rainfall as if it were a forecast. Fixing these in Week 4 protects every later result.

4. Detailed activities

- Define the forecast origin: end of day t, predicting the mean moisture of day t+1 (horizon 1) and optionally t+3.
- Create lag features, rolling rainfall sums, temperature and ET0 features and seasonal terms (day-of-year sine and cosine).
- Define the targets and drop rows without a valid target.
- Split chronologically with a purge gap equal to the horizon.
- Wrap preprocessing and model in a scikit-learn Pipeline fitted on training data only.

5. Step-by-step implementation procedure

1. Implement `add_lag_features`, `add_rolling_features`, `add_season_features` (see Section 6).
2. Implement `make_target(df, col, horizon)` using `shift(-horizon)`.
3. Choose split dates, for example train up to 2022, validation 2023, test 2024–2025, keep the most recent months for a final demo **[I]**; adjust to the data actually downloaded.
4. Remove the last *horizon* training rows so that no training target overlaps the validation period.
5. Build the leakage checklist test: for a synthetic series with known values, assert each feature at row t equals data from day t or earlier.
6. Create `make_dataset.py` that reads `data/processed`, writes `data/features/sm_features.csv` and a JSON describing features, target and split dates.
7. For the yield and crop datasets prepare their own splits (grouped by year; stratified by class) and record them.
8. Review the leakage checklist with a peer who did not write the code.

11. Testing and verification criteria

- Target at row t equals the moisture value at day t+horizon (unit test on a toy series).
- max(train date) < min(validation date) < min(test date).
- No feature uses a value dated after the forecast origin.
- Scalers or imputers are fitted only on training rows.

12. Common problems and solutions

Table 20. Week 4: common problems and suggested solutions

Problem	Solution
Many NaNs after lags	Expected for the first rows (longest lag); drop them after feature creation.
Unsure whether a feature leaks	Ask: would this exact value be known at the end of day t in real use? If not, remove it.
Observed rainfall is used for tomorrow	Use only rainfall up to day t as a feature; use forecast rainfall for t+1 only in the recommendation stage, and note the mismatch between reanalysis training data and forecasts.

13. Short explanation for the class

A model that sees the future gives impressive but meaningless scores. We split by time, shift targets carefully and test the pipeline for leakage before we train anything.

14. Suggested presentation slides and screenshots

- Figure: chronological split diagram
- Table: feature list with descriptions
- Code screenshot: shift and rolling features (real code)

15. Checklist

- ☐ Forecast origin and horizon defined
- ☐ Features implemented
- ☐ Targets implemented
- ☐ Chronological splits created
- ☐ Purge gap applied
- ☐ Leakage tests pass
- ☐ Feature table saved

Week 5: Soil-Moisture Forecasting

1. Objective	Train and evaluate a next-day soil-moisture model against a persistence baseline and save it for the application.
Suggested lead	Ravi Raj Kumar **[A]**
6. Software and tools	scikit-learn, pandas, NumPy, Plotly, joblib.
7. Dataset requirements	Daily feature table from DS-01.
8. Python files / modules	src/farmopt/irrigation_model.py (training and prediction), src/farmopt/evaluation.py, models/, reports/figures/.
9. Expected output	Trained model, metrics table and figures. Real numbers appear only after this week's run **[E]**.
10. Deliverables	Trained model and metadata; Metrics table versus baseline; Actual vs predicted plot; Short interpretation including limitations.

2–3. Technical background

Soil-moisture forecasting is a regression problem on a time series. The honest baseline is *persistence*: tomorrow equals today. Because moisture changes slowly, persistence is hard to beat on dry days; models typically add value around rainfall events and wetting or drying transitions. A Random Forest averages many decorrelated decision trees and handles non-linear interactions, mixed feature scales and modest data sizes without much tuning, but it cannot extrapolate beyond the range seen in training. Results describe how well the model reproduces the *reanalysis* soil-moisture series, not a real field.

4. Detailed activities

- Compute persistence (and optionally day-of-year climatology) baselines on validation and test sets.
- Train a Random Forest regressor; optionally compare HistGradientBoostingRegressor.
- Tune a few hyperparameters with `TimeSeriesSplit` inside the training period only.
- Evaluate MAE, RMSE and R^2 on validation, then once on the test set.
- Analyse errors by month and by rain versus no-rain days; plot actual versus predicted.
- Save the model and metadata with joblib.

5. Step-by-step implementation procedure

1. Load the feature table and split with the dates from Week 4.
2. Baseline: `y_pred = test['sm_0_7cm']` (value at day t used for t+1). Compute the three metrics.
3. Train `RandomForestRegressor(n_estimators=300, min_samples_leaf=3, random_state=42, n_jobs=-1)`.
4. Tune `max_depth`, `min_samples_leaf`, `max_features` with `TimeSeriesSplit(n_splits=5)` on the training set only.
5. Evaluate on the validation set, finalise choices, then evaluate once on the untouched test set.
6. Plots: actual vs predicted line chart for the test period, 1:1 scatter, residual histogram, MAE by month, feature importance (and permutation importance on validation).
7. Compare with the baseline using MAE, RMSE, R^2 and the skill score $1 - \mathrm{MAE}_{\mathrm{model}}/\mathrm{MAE}_{\mathrm{persistence}}$.
8. Save `models/sm_rf_v1.joblib` and `models/sm_rf_v1.json` (features, target, split dates, library versions, metrics).

11. Testing and verification criteria

- Saved model loads in a fresh process and reproduces predictions on a stored sample.
- Metrics are computed only on the test set that was never used for tuning.
- Baseline and model use identical rows.
- Re-running with the same seed gives the same metrics.

12. Common problems and solutions

Table 21. Week 5: common problems and suggested solutions

Problem	Solution
Model does not beat persistence	Report it honestly; check features, look at rain-event days separately, try gradient boosting or predicting the change in moisture instead of the level.
Very high R^2 but poor usefulness	R^2 on a highly autocorrelated series is inflated; always compare with persistence and report MAE.
Validation much better than test	Check for regime change (a wet or dry year) and for leakage; describe the difference.

13. Short explanation for the class

We forecast tomorrow's modelled soil moisture and compare against the simplest possible forecast, 'same as today'. We report MAE, RMSE and R^2 on a test period the model never saw.

14. Suggested presentation slides and screenshots

- Figure: actual vs predicted (real)
- Table: baseline vs Random Forest metrics (real)
- Figure: feature importance
- Slide: limitations of modelled moisture

15. Checklist

☐ Persistence baseline computed
☐ Random Forest trained
☐ Optional boosting compared
☐ TimeSeriesSplit tuning on train only
☐ Test evaluation done once
☐ Plots saved
☐ Model and metadata saved

Week 6: Irrigation Recommendation Engine

1. Objective	Convert the moisture forecast and rain outlook into transparent, explainable, crop-specific irrigation recommendations.
Suggested lead	Ravi Raj Kumar **[A]**
6. Software and tools	Python dataclasses, pandas, PyYAML, pytest.
7. Dataset requirements	Model forecasts from Week 5; Open-Meteo forecast API (or cached forecast) for the rain outlook.
8. Python files / modules	src/farmopt/recommendation_engine.py, config/crops.yaml, tests/test_recommendation.py.
9. Expected output	Function returning action, reason and optional indicative depth; log of daily recommendations.
10. Deliverables	recommendation_engine.py; Crop configuration with source notes; Rule documentation; Test report for wet, dry and missing cases.

2–3. Technical background

Irrigation decisions should be explainable to a farmer and a faculty panel, so the engine uses explicit rules rather than a black box. A common idea is to irrigate when the root zone has used a set fraction of its available water. With wilting-point moisture (θ_{wp}) and field-capacity moisture (θ_{fc}), relative saturation is $RSI = (\theta - \theta_{wp})/(\theta_{fc} - \theta_{wp})$, and irrigation is triggered when RSI drops below a crop-specific level. Rain expected within the forecast horizon can defer irrigation. How *long* to irrigate cannot be inferred from moisture alone: it needs the depth of water to apply, root depth, field area, flow rate and application efficiency.

4. Detailed activities

- Define the rule set and a configuration file of crop parameters.
- Estimate θ_{wp} and θ_{fc} proxies for the pilot location (for example low and high percentiles of the local series) and label them as proxies **[A]**.
- Implement rain deferral and missing-data handling.
- Produce a plain-language explanation for every outcome.
- Test wet, dry and missing-data cases.

5. Step-by-step implementation procedure

1. Create `config/crops.yaml` with, per crop, `trigger_rsi`, `rain_defer_mm`, `root_depth_cm` and a source note. Values must come from a cited agronomic reference chosen by the team; until then mark them as placeholders **[I]**.
2. Compute θ_{wp} and θ_{fc} proxies for the location, or let the user override them with soil-test values.
3. Implement `recommend(...)` returning action, reason and the inputs used (flowchart below).
4. Add an optional `irrigation_depth_mm` = $(\theta_{target} - \theta_{now}) \times$ root depth (mm) when root depth is known; add volume and time only if area, flow rate and efficiency are provided.
5. Write unit tests: very wet soil (NO_IRRIGATION), dry soil without rain (IRRIGATE), dry soil with forecast rain (DEFER), NaN moisture (NO_RECOMMENDATION), invalid parameters.
6. Run the engine over the test period using the model forecasts and tabulate the daily actions.
7. Review the explanation texts with a non-technical reader.

11. Testing and verification criteria

- Each branch of the flowchart has a unit test.
- Missing or NaN input never produces IRRIGATE.
- Changing the rain threshold changes the outcome in the expected direction.
- Every output contains a reason string.

12. Common problems and solutions

Table 22. Week 6: common problems and suggested solutions

Problem	Solution
Thresholds are arbitrary	State that they are configurable, cite the source used, and show a sensitivity analysis in Week 9.
Rain forecast says 'rain' but none falls	Use forecast probability and amount, re-evaluate daily, and allow a stress override when RSI is critically low.
Reviewers ask for minutes of irrigation	Explain that duration needs flow rate, area, root depth and efficiency; offer the optional calculator with user-supplied parameters.

13. Short explanation for the class

The engine is deliberately transparent: it compares forecast soil water with a crop-specific trigger, checks whether rain is coming, and always tells the user why. It gives a duration only when the user supplies system details.

14. Suggested presentation slides and screenshots

- Figure: decision flowchart
- Table: crop parameters with source
- Screenshot: recommendation with reason text (real, after build)

15. Checklist

- ☐ Crop config created
- ☐ θwp/θfc proxies computed and labelled
- ☐ recommend() implemented
- ☐ Reasons generated
- ☐ Wet/dry/missing tests pass
- ☐ Duration formula documented

Week 7: Crop Suitability and Fertilizer Guidance

1. Objective	Train and evaluate a crop classifier and implement documented, rule-based nutrient guidance.
Suggested lead	Kartik Sharma **[A]**
6. Software and tools	scikit-learn, pandas, Plotly, PyYAML, joblib.
7. Dataset requirements	DS-03 (classifier); DS-07 user-entered soil-test values (rules); optional DS-08 for exploration only.
8. Python files / modules	src/farmopt/crop_model.py, src/farmopt/fertilizer_recommendation.py, config/nutrient_rules.yaml, models/crop_rf_v1.joblib.
9. Expected output	Classifier metrics and confusion matrix **[E]**; rule-based guidance function.
10. Deliverables	Crop classifier and metrics; Confusion matrix and per-class report; Nutrient rule table with sources; Statement of limitations.

2–3. Technical background

Crop recommendation is a multi-class classification problem: seven numeric inputs map to one of 22 crop labels in the Kaggle dataset. Fertiliser-dose prediction is a different problem. It needs a numeric dose label that depends on crop, soil test, yield target and local recommendations, and public data rarely supply it. Treating a crop label as if it were a dose would be wrong, so the team uses a classifier for crop suitability and a rule table, sourced from an official recommendation, for nutrient guidance.

4. Detailed activities

- Explore class balance and feature distributions; check duplicates.
- Create a stratified 80/20 split and a stratified 5-fold cross-validation.
- Train baselines (majority class, Decision Tree, KNN) and a Random Forest.
- Evaluate accuracy, macro precision, recall and F1, per-class report and confusion matrix.
- Return the top three crops with probabilities for the dashboard.
- Implement rule-based nutrient guidance using lab-report units.

5. Step-by-step implementation procedure

1. Load the crop CSV, assert columns and ranges, drop exact duplicates.
2. `train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)`.
3. Baseline: `DummyClassifier(strategy='most_frequent')`; then Decision Tree, KNN (with StandardScaler in a Pipeline) and Random Forest.
4. Cross-validate with `StratifiedKFold(5, shuffle=True, random_state=42)`; report mean and standard deviation.
5. Final evaluation on the held-out 20 %: accuracy, macro-F1, classification report, confusion matrix plot.
6. Use `predict_proba` to show the three most likely crops.
7. Create `config/nutrient_rules.yaml`: per crop, low/adequate/high bands for N, P, K and a preferred pH range, each with a source note (state agricultural university, ICAR or Soil Health Card guidance chosen by the team) **[A]**.
8. Implement `nutrient_guidance(soil_test, crop)` returning a status per nutrient and advisory text, always ending with a note to confirm with local extension advice.

11. Testing and verification criteria

- Stratification gives each class the same proportion in train and test.
- Classifier accepts only the seven expected features in the expected order.
- Nutrient function returns an explicit 'unknown crop' result for crops without rules.
- No unit conversion is performed between dataset N/P/K and lab values.

12. Common problems and solutions

Table 23. Week 7: common problems and suggested solutions

Problem	Solution
Accuracy near 99 %	Likely because classes are well separated in this compiled dataset. Report it, but state that it does not prove real-field performance.
No dose labels available	Keep guidance rule-based and cite the rule source; do not train a dose model on invented labels.
Unit confusion between dataset and soil-test values	Keep two separate input forms and pipelines; document units in the data dictionary.

13. Short explanation for the class

Choosing a crop is a classification task and we evaluate it with accuracy and F1. Fertiliser dose is different: it needs agronomic rules and soil-test units, so we implement documented rules rather than pretending a model can predict doses.

14. Suggested presentation slides and screenshots

- Table: classifier comparison (real)
- Figure: confusion matrix (real)
- Slide: crop recommendation vs fertiliser guidance

15. Checklist

☐ Class balance checked
☐ Stratified split done
☐ Baseline and models trained
☐ Metrics and confusion matrix saved
☐ Top-3 output implemented
☐ Nutrient rules sourced
☐ Guidance function tested

Week 8: Crop-Yield Prediction

1. Objective	Build a district-level yield model on actual historical yield data with honest baselines and temporal evaluation.
Suggested lead	Kartik Sharma **[A]**
6. Software and tools	pandas, scikit-learn, Open-Meteo Geocoding and Historical APIs, Plotly.
7. Dataset requirements	DS-04 (primary), DS-05 (cross-check), DS-01 (weather join, optional).
8. Python files / modules	src/farmopt/yield_model.py, config/districts.csv (optional), models/yield_rf_v1.joblib.
9. Expected output	Yield model metrics **[E]** and error analysis by crop and state.
10. Deliverables	yield_model.py; Baseline comparison table; Temporal and geographic evaluation; Limitations paragraph.

2–3. Technical background

Yield is the output of weather, soil, crop variety and management. Public data give district-level area and production by crop, season and year, but no soil or management information; therefore the model can capture crop, region, season, long-term trend and (if joined) seasonal weather, and nothing more. Yield equals production divided by area. A fair test holds out later years, because the model will be used for future seasons; an additional leave-district-out test checks geographic transfer. Baselines include the previous year's yield and the crop-district historical mean.

4. Detailed activities

- Select crops (rice, wheat, maize) and a state or a few states to keep the problem manageable.
- Derive yield, clean units and outliers.
- Optionally join seasonal weather aggregates (total rainfall, mean temperature) per district-year-season from Open-Meteo using district coordinates.
- Train baselines, Ridge regression, Random Forest and optionally HistGradientBoostingRegressor.
- Evaluate MAE, RMSE and R^2 on held-out years and on held-out districts.
- Document geographic and temporal limits.

5. Step-by-step implementation procedure

1. Compute `yield_t_ha = production / area` after unit checks; drop zero area.
2. Filter to chosen crops and states; encode state, district, crop and season (one-hot or ordinal for tree models).
3. Add a year (trend) feature to capture technology and management trends.
4. Optional weather join: geocode districts (Open-Meteo Geocoding API) or maintain a manual latitude/longitude table; request seasonal aggregates; cache locally.
5. Chronological split, for example train 1997–2010, validation 2011–2012, test 2013–2015 **[I]**; adjust to the data.
6. Baselines: previous-year yield for the same district-crop-season; mean of training years for the district-crop.
7. Train models; tune on validation; evaluate once on the test years; repeat with a leave-district-group-out test.
8. Write the interpretation: where errors are large, which crops are worse, and what cannot be concluded.

11. Testing and verification criteria

- No test-year row appears in training.
- Yield derivation unit test with known area and production.
- Model rejects unseen categories gracefully (handle_unknown).
- Predictions are never presented for farms, only for district-crop-season combinations.

12. Common problems and solutions

Table 24. Week 8: common problems and suggested solutions

Problem	Solution
Yield values differ by orders of magnitude between crops	Model by crop or include crop as a feature and evaluate per crop; consider log-yield with care.
Dataset ends in 2015	State it. Use it for methodology demonstration; use ICRISAT or newer official statistics if recent years are needed.
Weather join is slow	Cache per district-year, reduce to a few districts, or skip the join and report a model without weather.

13. Short explanation for the class

We predict district-level yield from crop, region, season and trend (and optionally seasonal weather) and test it on later years. We do not claim farm-level prediction because public data do not contain farm information.

14. Suggested presentation slides and screenshots

- Table: baselines vs models (real)
- Figure: predicted vs actual yield (real)
- Slide: district vs state vs farm level

15. Checklist

☐ Yield derived and checked
☐ Crops and states selected
☐ Temporal split done
☐ Baselines computed
☐ Models trained
☐ Leave-district-out test
☐ Limitations written

Week 9: Resource Optimization

1. Objective	Compare a fixed-schedule irrigation baseline with a prediction-informed policy in a transparent simulation, without claiming real savings.
Suggested lead	Ravi Raj Kumar **[A]**
6. Software and tools	pandas, NumPy, Plotly, PyYAML.
7. Dataset requirements	DS-01 (historical rain and ET0 for the test period), DS-09 scenario parameters.
8. Python files / modules	src/farmopt/optimization.py, config/scenario.yaml, reports/tables/resource_comparison.csv.
9. Expected output	Scenario comparison table and charts [E, simulated].
10. Deliverables	optimization.py; Scenario configuration; Comparison table with stress days; Sensitivity analysis; Assumptions and limitations text.

2–3. Technical background

The report requires evaluating resource use against a fixed-interval control **[R]**. Without a field trial, the only honest option is a simulation: a daily soil-water balance (bucket model) driven by historical rainfall and evapotranspiration. Soil water changes as $S_{t+1} = S_t$ + effective rain + irrigation − crop evapotranspiration − drainage, bounded between wilting point and field capacity. Crop evapotranspiration is $ET_c = K_c \times ET_0$ with a crop coefficient K_c. Results depend entirely on these assumptions, so they are labelled as simulated scenarios and always shown with stress days beside water use.

4. Detailed activities

- Specify one scenario: crop, season, root depth, soil water limits, irrigation depth per event, application efficiency.
- Implement the daily water-balance simulator.
- Implement three policies: fixed interval, threshold only, and threshold with rain deferral using forecast moisture.
- Compute water applied, number of events, stress days and drainage loss.
- Run sensitivity analysis on thresholds, interval length and forecast error.
- Document assumptions and limitations.

5. Step-by-step implementation procedure

1. Create `config/scenario.yaml` with every parameter and a source note or the label "assumption".
2. Implement `simulate(weather, policy, params)` that returns a daily table of soil water, irrigation, ET_c, drainage and stress flags.
3. Fixed policy: apply depth *D* every *N* days regardless of conditions.
4. Informed policy: call `recommend()` each day with the model forecast and a rain outlook.
5. Rain outlook: avoid oracle forecasts. Use forecast values (if archived forecasts are available from the Open-Meteo historical forecast service, check docs) or add random forecast error and show how results change.
6. Compute: total irrigation (mm), events, stress days (RSI below a critical level), days of excess drainage.
7. Percentage water difference = $(W_{fixed} - W_{policy}) / W_{fixed} \times 100$, reported only together with stress days; if the policy has more stress days the saving is not comparable.
8. Export `reports/tables/resource_comparison.csv` with columns marked "simulated".

11. Testing and verification criteria

- Water balance conserves mass in a toy example (hand-calculated).
- Fixed policy irrigates on the expected days.
- Soil water never exceeds field capacity or falls below zero.
- Output files carry a 'simulated' label.

12. Common problems and solutions

Table 25. Week 9: common problems and suggested solutions

Problem	Solution
Informed policy wins by a huge margin	Check for oracle rainfall, a poorly chosen fixed interval, or hidden stress; compare against several reasonable fixed schedules.
Stress days appear under the informed policy	Lower the trigger or add a stress override; report the trade-off honestly.
Reviewers ask for real savings	Explain that field validation with metered irrigation is future work.

13. Short explanation for the class

We simulate two ways of watering the same field: on a calendar or based on forecasts. We compare estimated water use, events and stress days under written assumptions. This is a simulation, so we do not claim real-world savings.

14. Suggested presentation slides and screenshots

- Table: resource comparison (simulated)
- Figure: soil-water trajectory under two policies
- Slide: assumptions and limitations

15. Checklist

- ☐ Scenario parameters written
- ☐ Simulator tested by hand
- ☐ Three policies implemented
- ☐ Metrics computed
- ☐ Sensitivity analysis done
- ☐ Simulated label on outputs
- ☐ Assumptions documented

Week 10: Dashboard Development

1. Objective	Build a modular Streamlit dashboard that integrates saved models and supports offline operation.
Suggested lead	Prashant Kumar Yadav **[A]**
6. Software and tools	Streamlit, Plotly, pandas, joblib, requests.
7. Dataset requirements	Local processed data, saved models and sample data; optional live APIs.
8. Python files / modules	app.py, dashboard/overview.py, soil_weather.py, irrigation.py, crop_fertilizer.py, yield_page.py, evaluation_page.py, scenario.py.
9. Expected output	Working local web application (screenshots captured from the real app).
10. Deliverables	Dashboard source code; Run instructions; Real screenshots of every page; Offline-mode demonstration.

2–3. Technical background

A dashboard makes the work understandable to farmers and examiners. Streamlit re-runs the script on each interaction, so models and data should be loaded once with caching (`st.cache_resource` for models, `st.cache_data` for data). The app is a thin router; each page is a function in its own module, and all logic stays in `src/farmopt`. Every page shows data provenance, units and a note that modelled values are not sensor readings.

4. Detailed activities

- Create the app skeleton and navigation.
- Implement the eight pages from the design in Section 9.
- Add selectors, charts, explanations and CSV downloads.
- Integrate the saved models and the recommendation engine.
- Implement online/offline mode with local fallback.
- Handle errors with clear messages instead of tracebacks.

5. Step-by-step implementation procedure

1. Create `app.py` with `st.set_page_config` and a sidebar navigation that calls `dashboard/*.py` page functions.
2. Write `load_daily_data(mode)`: try the API when online, otherwise read the cache or `data/sample`; return the data and a mode flag shown in the sidebar.
3. Soil and weather page: Plotly time series, date range selector, provenance badges.
4. Irrigation page: location/crop selectors, forecast with trigger line, recommendation card and reason text.
5. Crop and fertiliser pages: input forms with validation, top-3 crops, nutrient status table.
6. Yield page: crop, district, season selectors; prediction with historical context and a limitation note.
7. Evaluation page: load metrics JSON and figures; show baseline versus model tables.
8. Scenario page: parameters panel and comparison charts labelled "simulated".
9. Add `st.download_button` for CSV exports and test the app with the network disabled.

11. Testing and verification criteria

- App starts with `streamlit run app.py` on a clean environment.
- Each page renders with the network disabled.
- Invalid inputs show a helpful message.
- Downloaded CSV opens correctly and has the documented columns.

12. Common problems and solutions

Table 26. Week 10: common problems and suggested solutions

Problem	Solution
App is slow after each click	Cache model loading and data reads; avoid retraining in the app.
API failure crashes a page	Wrap calls in try/except, fall back to local data and show an 'offline' badge.
Page code grows large	Keep logic in src/farmopt; pages only gather inputs and display results.

13. Short explanation for the class

The dashboard is the farmer-facing layer: pick a location and crop, see the forecast, the recommendation and the reason. It runs offline from local data and always shows where each number comes from.

14. Suggested presentation slides and screenshots

- Screenshot of each page (real)
- Figure: wireframe versus final screen
- Slide: offline mode demonstration

15. Checklist

- ☐ App skeleton running
- ☐ Eight pages built
- ☐ Models integrated
- ☐ Downloads work
- ☐ Offline fallback works
- ☐ Errors handled
- ☐ Screenshots captured

Week 11: Testing and Model Evaluation

1. Objective	Verify data processing, models, rules, error handling and reproducibility; record limitations and bugs.
Suggested lead	Prashant Kumar Yadav with all members **[A]**
6. Software and tools	pytest, pytest monkeypatch, pandas, Git, (optional) pytest-cov.
7. Dataset requirements	Test fixtures (synthetic), sample data, processed data.
8. Python files / modules	tests/ (test_preprocessing.py, test_features.py, test_models.py, test_rules.py, test_optimization.py, test_app_smoke.py), docs/known_issues.md.
9. Expected output	Passing test suite, frozen evaluation tables, bug and limitation log.
10. Deliverables	Test suite and report; Reproducibility check log; Final metric tables; Known issues and limitations.

2–3. Technical background

Three kinds of assessment are easy to confuse. *Model metrics* (MAE, RMSE, R^2, accuracy, F1) measure predictive error on held-out data. *Application tests* (pytest, manual UI checks) measure whether the software behaves as specified. *Real-world agricultural impact* (water or yield change in a field) can only be measured in a field trial, which this project does not have. Week 11 completes the first two and states clearly that the third is out of scope.

4. Detailed activities

- Complete the test suite for preprocessing, features, models, rules and optimization.
- Simulate API failures and offline operation.
- Validate the train/validation/test split and the absence of leakage.
- Check reproducibility from a clean environment.
- Run the evaluation scripts and freeze the final metric tables.
- Log bugs and limitations.

5. Step-by-step implementation procedure

1. List all tests in the matrix of Section 10 and assign each to an owner.
2. Write pytest tests with small synthetic fixtures; mark slow tests.
3. API failure tests: monkeypatch `requests.get` to raise `requests.ConnectionError` and assert the fallback path is used.
4. Run `pytest -q` locally and fix failures; record the pass count.
5. Reproducibility: create a new venv, install pinned requirements, run the training scripts twice and compare metrics and file hashes.
6. Re-run evaluation scripts to regenerate `reports/tables/*.csv` and figures.
7. Manual UI test script: each page, valid and invalid input, offline mode.
8. Write `docs/known_issues.md` with bugs, limitations and workarounds.

11. Testing and verification criteria

- All tests pass on a clean environment.
- Training twice with the same seed gives identical metrics.
- Offline test passes with the network disabled.
- Final metric tables are generated by scripts, not typed by hand.

12. Common problems and solutions

Table 27. Week 11: common problems and suggested solutions

Problem	Solution
Tests pass locally but fail on another machine	Pin versions, avoid absolute paths, use pathlib and relative paths.
Non-reproducible metrics	Set `random_state` everywhere and sort data deterministically; record library versions.
Flaky API tests	Never call the real API in unit tests; mock the response.

13. Short explanation for the class

We separate three things: model metrics on held-out data, software tests that check the code, and real-world impact, which needs field trials and is future work.

14. Suggested presentation slides and screenshots

- Table: testing matrix with pass/fail (real)
- Screenshot: pytest summary (real)
- Slide: three levels of evaluation

15. Checklist

- ☐ Preprocessing tests
- ☐ Feature and leakage tests
- ☐ Model load/predict tests
- ☐ Rule tests
- ☐ API failure tests
- ☐ Reproducibility check
- ☐ Known issues logged

Week 12: Final Integration and Presentation

1. Objective	Freeze the code, finish documentation, capture real screenshots, prepare tables, slides and viva answers, and rehearse the demonstration.
Suggested lead	All members; integration by Prashant Kumar Yadav **[A]**
6. Software and tools	Git, Streamlit, a screen recorder, presentation software, the final report template.
7. Dataset requirements	Frozen processed data and sample data for offline demo.
8. Python files / modules	README.md, docs/final_report.md, reports/, slides/, release v1.0.
9. Expected output	Tagged repository, final report, slide deck, demo video, viva notes.
10. Deliverables	Final source code (tagged); Documentation and README; Real screenshots and evaluation tables; Presentation deck; Rehearsal log and viva Q&A.

2–3. Technical background

The last week turns working code into a defensible submission. A tagged release, a README that lets a stranger run the project, evaluation tables produced by scripts, and a rehearsed demonstration with an offline backup protect against last-minute failures. Slides must separate completed work from planned work and use real screenshots and real numbers only.

4. Detailed activities

- Merge branches, freeze the code and tag release v1.0.
- Complete README, run instructions and final report sections.
- Capture real screenshots and generate final tables.
- Prepare the 15-slide presentation and assign presenters.
- Rehearse the demo (online and offline) at least twice.
- Prepare viva answers (Section 14).

5. Step-by-step implementation procedure

1. Code freeze meeting; merge and resolve conflicts; run the full test suite.
2. Tag the release (`git tag v1.0`) and export a clean copy.
3. Update README with install, data download, training and run commands.
4. Run all scripts to regenerate tables and figures; copy them into the report.
5. Capture screenshots of every page from the running app (record date and data mode).
6. Build the slides (Section 13); label each item Completed or Planned.
7. Assign presenters by topic; rehearse with a timer; record a backup demo video.
8. Mock viva: each member answers questions outside their own module.

11. Testing and verification criteria

- A fresh clone follows the README and starts the dashboard.
- All numbers in the slides trace to files in reports/.
- The offline demo runs without internet.
- Every member can explain every module at overview level.

12. Common problems and solutions

Table 28. Week 12: common problems and suggested solutions

Problem	Solution
Demo fails on the day	Keep the offline sample data, a recorded video and a screenshot set as backups.
Numbers in slides differ from the report	Generate both from the same CSV files.
Time overrun	Rehearse with a timer; keep appendix slides for details.

13. Short explanation for the class

Here is what we planned, what we built and what we measured, with clear labels. Everything runs offline, and the limitations are stated.

14. Suggested presentation slides and screenshots

- All 15 slides from Section 13
- Live demo and backup video
- Appendix: test report and limitations

15. Checklist

- ☐ Code frozen and tagged
- ☐ README verified by a fresh clone
- ☐ Final tables generated
- ☐ Screenshots captured
- ☐ Slides finished
- ☐ Presenters assigned
- ☐ Two rehearsals done
- ☐ Viva practice done

6 Technical Implementation Guide

This section shows how the proposed stack fits together [A]. Snippets are short, illustrative and written for Python 3.10 or newer with pandas, NumPy, scikit-learn, requests, Streamlit, Plotly, joblib and pytest. They have not been run for this document; the team must run and test them. Check each library's current documentation before relying on an API.

Table 29. Technology stack

Tool	Role in the project
Python 3.10+	Single language for data, models and app.
pandas, NumPy	Tables, time-series resampling, numerical work.
scikit-learn	Pipelines, Random Forest, HistGradientBoosting, metrics, TimeSeriesSplit.
requests	HTTP calls to Open-Meteo (with timeouts and error handling).
Streamlit, Plotly	Dashboard and interactive charts.
joblib	Saving and loading trained models.
pytest	Unit and smoke tests.
CSV, optional SQLite	Local storage; SQLite (standard library `sqlite3`) only if a query layer is wanted.

6.1 Recommended folder structure

Project layout (data and models are excluded from Git by .gitignore)

```
farm-resource-optimizer/
  README.md   requirements.txt   config.yaml   app.py
  config/        crops.yaml  nutrient_rules.yaml  scenario.yaml
  data/          raw/  interim/  processed/  features/  sample/
  models/        *.joblib  *.json        (trained models + metadata)
  reports/       figures/  tables/  data_quality.md
  src/farmopt/
      __init__.py
      data_collection.py   preprocessing.py   features.py
      irrigation_model.py  crop_model.py      yield_model.py
      fertilizer_recommendation.py  recommendation_engine.py
      optimization.py      evaluation.py
  dashboard/     overview.py  soil_weather.py  irrigation.py
                 crop_fertilizer.py  yield_page.py  evaluation_page.py  scenario.py
  scripts/       make_dataset.py  train_all.py  evaluate_all.py
  tests/         test_*.py
  notebooks/     01_eda.ipynb ...
  docs/          dataset_register.csv  traceability.csv  known_issues.md
```

Table 30. Responsibilities of the main files

File	Responsibility	Must not contain
data_collection.py	Fetch and cache Open-Meteo data; load Kaggle/ICRISAT files; write manifest and metadata; offline fallback.	Cleaning or modelling logic.
preprocessing.py	Schema and range validation, timestamp parsing, hourly-to-daily aggregation, missing and duplicate handling.	Feature creation.
features.py	Lags, rolling sums, seasonal terms, targets, chronological splits.	Model training.
irrigation_model.py	Train, evaluate, save, load and predict the soil-moisture model.	Irrigation decision rules.
crop_model.py	Crop classifier training and prediction (top-k).	Fertiliser rules.
fertilizer_recommendation.py	Rule-based nutrient guidance from the YAML rule table.	Any ML model.
yield_model.py	District-level yield regression and baselines.	Dashboard code.
recommendation_engine.py	Combine forecast, rain outlook and crop rules into explained actions.	Data download.
optimization.py	Water-balance simulation and policy comparison.	Streamlit calls.
evaluation.py	Metrics (MAE, RMSE, R^2, classification), tables and plots.	Training.
app.py	Thin router: configuration, navigation, calling page functions.	Business logic.

6.2 Illustrative snippets

Collection with cache and offline fallback (check variable names on the Open-Meteo docs page)

```
# data_collection.py  (illustrative)
import requests, pandas as pd
from pathlib import Path

ARCHIVE_URL = "https://archive-api.open-meteo.com/v1/archive"
HOURLY = ("soil_moisture_0_to_7cm,soil_moisture_7_to_28cm,soil_moisture_28_to_100cm,"
          "temperature_2m,relative_humidity_2m,precipitation,et0_fao_evapotranspiration")

def fetch_archive_hourly(lat, lon, start, end, timeout=30):
    params = {"latitude": lat, "longitude": lon, "start_date": start, "end_date": end,
              "hourly": HOURLY, "timezone": "Asia/Kolkata"}
    r = requests.get(ARCHIVE_URL, params=params, timeout=timeout)
    r.raise_for_status()
    df = pd.DataFrame(r.json()["hourly"])
    df["time"] = pd.to_datetime(df["time"])
    return df.set_index("time")

def load_weather(lat, lon, start, end, cache_dir="data/raw/openmeteo"):
    path = Path(cache_dir) / f"archive_{lat:.3f}_{lon:.3f}_{start}_{end}.csv"
    try:
        df = fetch_archive_hourly(lat, lon, start, end)
        path.parent.mkdir(parents=True, exist_ok=True)
        df.to_csv(path)
        return df, "online"
    except requests.RequestException:
        if path.exists():
            return pd.read_csv(path, index_col=0, parse_dates=True), "cache"
        raise FileNotFoundError(f"No internet and no cached file at {path}")
```

Aggregation and validation

```
# preprocessing.py  (illustrative)
import pandas as pd

def to_daily(hourly: pd.DataFrame) -> pd.DataFrame:
    agg = {"soil_moisture_0_to_7cm": "mean", "soil_moisture_7_to_28cm": "mean",
           "soil_moisture_28_to_100cm": "mean", "temperature_2m": "mean",
           "relative_humidity_2m": "mean", "precipitation": "sum",
           "et0_fao_evapotranspiration": "sum"}
    daily = hourly.resample("D").agg(agg)
    return daily.rename(columns={"soil_moisture_0_to_7cm": "sm_0_7cm",
                                 "soil_moisture_7_to_28cm": "sm_7_28cm",
                                 "soil_moisture_28_to_100cm": "sm_28_100cm",
                                 "temperature_2m": "tmean_c", "relative_humidity_2m": "rh_mean",
                                 "precipitation": "rain_mm",
                                 "et0_fao_evapotranspiration": "et0_mm"})

def check_ranges(d: pd.DataFrame) -> list[str]:
    problems = []
    if not d["sm_0_7cm"].dropna().between(0, 1).all(): problems.append("moisture outside 0-1")
    if (d["rain_mm"].dropna() < 0).any(): problems.append("negative rainfall")
    if d.index.has_duplicates: problems.append("duplicate dates")
    return problems
```

Lag, rolling and target features with a chronological split

```
# features.py  (illustrative)
import numpy as np

def add_features(df, col="sm_0_7cm", horizon=1):
    out = df.copy()
    for k in (1, 2, 3, 7):
        out[f"{col}_lag{k}"] = out[col].shift(k)             # past values only
    for w in (3, 7, 14):
        out[f"rain_sum{w}d"] = out["rain_mm"].rolling(w, min_periods=w).sum()  # days t-w+1..t
    doy = out.index.dayofyear
    out["doy_sin"] = np.sin(2 * np.pi * doy / 365.25)
    out["doy_cos"] = np.cos(2 * np.pi * doy / 365.25)
    out["target"] = out[col].shift(-horizon)                  # value at day t+horizon
    return out.dropna()

def chrono_split(df, train_end, val_end, horizon=1):
    train = df[df.index <= train_end].iloc[:-horizon]         # purge gap
    val = df[(df.index > train_end) & (df.index <= val_end)]
    test = df[df.index > val_end]
    return train, val, test
```

Metrics, Random Forest training and baseline

```
# irrigation_model.py  (illustrative)
import numpy as np, joblib
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

def metrics(y_true, y_pred):
    return {"MAE": mean_absolute_error(y_true, y_pred),
            "RMSE": float(np.sqrt(mean_squared_error(y_true, y_pred))),
            "R2": r2_score(y_true, y_pred)}

def train_rf(train, features, seed=42):
    rf = RandomForestRegressor(n_estimators=300, min_samples_leaf=3, random_state=seed, n_jobs=-1)
    return rf.fit(train[features], train["target"])

# persistence baseline: the forecast for day t+1 is the observed value on day t
# baseline = metrics(test["target"], test["sm_0_7cm"])
# joblib.dump({"model": rf, "features": features}, "models/sm_rf_v1.joblib")
```

Transparent rule engine

```
# recommendation_engine.py  (illustrative; thresholds are placeholders to be sourced)
import math
from dataclasses import dataclass

@dataclass(frozen=True)
class CropRule:
    name: str
    trigger_rsi: float      # irrigate when relative saturation falls below this
    rain_defer_mm: float    # defer when forecast rain reaches this amount

def rsi(theta, wp, fc):
    return (theta - wp) / (fc - wp)

def recommend(theta_fc, rain_mm, rule, wp, fc):
    bad = [v is None or (isinstance(v, float) and math.isnan(v)) for v in (theta_fc, rain_mm)]
    if any(bad) or fc <= wp:
        return {"action": "NO_RECOMMENDATION",
                "reason": "Missing or invalid moisture, rain or soil parameters."}
    s = rsi(theta_fc, wp, fc)
    if rain_mm >= rule.rain_defer_mm:
        return {"action": "DEFER",
                "reason": f"Forecast rain {rain_mm:.1f} mm reaches {rule.rain_defer_mm} mm."}
    if s < rule.trigger_rsi:
        return {"action": "IRRIGATE",
                "reason": f"Relative saturation {s:.2f} is below {rule.trigger_rsi} for {rule.name}."}
    return {"action": "NO_IRRIGATION",
            "reason": f"Relative saturation {s:.2f} is adequate for {rule.name}."}
```

Streamlit router and a download button

```
# app.py  (illustrative thin router)
import streamlit as st
from dashboard import (overview, soil_weather, irrigation, crop_fertilizer,
                       yield_page, evaluation_page, scenario)

st.set_page_config(page_title="Farm Resource Optimizer", layout="wide")
PAGES = {"Overview": overview.show, "Soil and weather": soil_weather.show,
         "Irrigation planner": irrigation.show, "Crop and fertilizer": crop_fertilizer.show,
         "Yield prediction": yield_page.show, "Model evaluation": evaluation_page.show,
         "Scenario simulator": scenario.show}
choice = st.sidebar.radio("Page", list(PAGES))
PAGES[choice]()
st.sidebar.caption("Weather data by Open-Meteo.com. Modelled values are not sensor readings.")

# inside a page: st.download_button("Download CSV", df.to_csv(index=False).encode("utf-8"),
#                                   file_name="recommendations.csv", mime="text/csv")
```

Leakage and split tests

```
# tests/test_features.py  (illustrative)
import pandas as pd
from farmopt.features import add_features, chrono_split

def toy():
    idx = pd.date_range("2020-01-01", periods=60, freq="D")
    return pd.DataFrame({"sm_0_7cm": range(60), "rain_mm": 1.0}, index=idx)

def test_target_is_next_day():
    df = add_features(toy())
    assert (df["target"] == df["sm_0_7cm"] + 1).all()

def test_split_is_chronological():
    tr, va, te = chrono_split(add_features(toy()), "2020-02-01", "2020-02-15")
    assert tr.index.max() < va.index.min() and va.index.max() < te.index.min()
```

Reproducibility

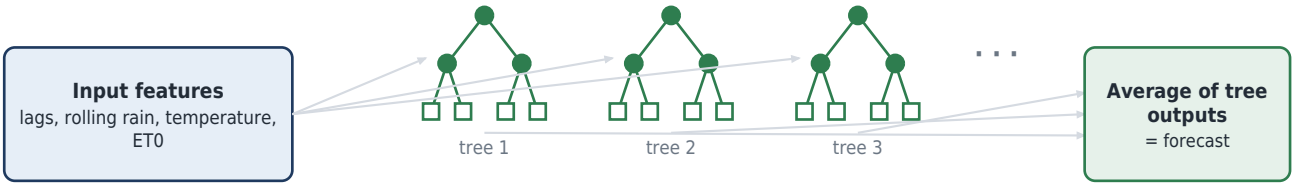
Pin versions with `pip freeze > requirements.txt`, set `random_state` everywhere, store feature lists with each model, and never retrain inside the dashboard.

7 Machine Learning Methodology

Each model below is specified with the same ten attributes: problem type, inputs, target, data requirements, candidate algorithms, baseline, validation strategy, metrics, output format and limitations [A]. Results are placeholders until real runs exist.

7.1 Soil-moisture regression

Problem type	Supervised regression on a daily time series (forecast horizon 1 day; optional 3 days).
Input features	Lagged moisture (1, 2, 3, 7 days), rolling rainfall (3, 7, 14 days), temperature, humidity, ET0, day-of-year sine/cosine.
Target	Daily mean soil moisture (m^3/m^3) at day t+h for the chosen layer or depth-weighted root zone.
Data requirements	Several years of daily data (suggested 8 or more) so that seasons repeat; documented units; few gaps.
Candidates	Persistence; day-of-year climatology; Random Forest; HistGradientBoostingRegressor (optional); Ridge (optional).
Baseline	Persistence: forecast(t+1) = observed(t).
Validation	Chronological train/validation/test with a purge gap; TimeSeriesSplit inside the training period for tuning.
Metrics	MAE, RMSE, R^2; skill versus persistence; errors split by rain and no-rain days.
Output format	Daily forecast table: date, predicted moisture, model version, provenance label.
Limitations	Learns a modelled series; grid-cell scale; cannot represent a field's irrigation history; may not extrapolate beyond training range.



Each tree is trained on a bootstrap sample with random feature subsets; regression output = mean of tree predictions.

Figure 5. *Random Forest regression schematic: many trees trained on different samples are averaged to give a forecast.*

7.2 Crop classification

Problem type	Multi-class classification.
Input features	N, P, K, temperature, humidity, pH, rainfall (dataset units).
Target	Crop label (22 classes in the Kaggle dataset).
Data requirements	Labelled rows with balanced or weighted classes; documented feature ranges.
Candidates	Decision Tree, KNN (scaled), Random Forest, optional gradient boosting.
Baseline	Most-frequent class (about 1/22 accuracy on a balanced set) and a simple single-feature rule.
Validation	Stratified 80/20 split and stratified 5-fold cross-validation (no time order exists).
Metrics	Accuracy, macro precision, macro recall, macro F1, confusion matrix, per-class report.
Output format	Top-3 crops with probabilities and a note that these are suitability suggestions.
Limitations	Compiled dataset with no location or season; high scores may not generalise; says nothing about fertiliser dose or profit.

7.3 Crop-yield regression

Problem type	Supervised regression on panel data (district × crop × season × year).

Input features	Crop, season, state, district, year (trend) and optional seasonal weather aggregates.
Target	Yield = production / area (for example tonne/ha) from actual statistics.
Data requirements	Many years per district-crop; consistent units; documented district boundaries.
Candidates	Ridge, Random Forest, HistGradientBoostingRegressor.
Baselines	Previous-year yield; district-crop training mean.
Validation	Hold out the latest years; additionally leave-district-group-out.
Metrics	MAE, RMSE, R^2; per-crop MAE.
Output format	Predicted district-level yield with the historical range for context.
Limitations	District-level only; no soil, variety or management data; data may end years ago; reporting errors in public statistics.

7.4 Rule-based irrigation recommendation

This component is **not** a trained model. It applies documented rules to model outputs. Inputs: forecast moisture, rain outlook, crop and soil parameters. Output: NO_RECOMMENDATION, DEFER, IRRIGATE or NO_IRRIGATION with a reason. It is verified by unit tests, not by accuracy metrics. Thresholds are configuration values with cited sources.

Indicative irrigation depth (mm) = ($\theta_{target} - \theta_{now}$) × root depth (mm); volume (L) = depth (mm) × area (m^2); time (h) = volume / (flow rate × efficiency). These need user-supplied area, flow rate and efficiency; without them, only the action is given.

7.5 Scenario-based resource comparison

A daily water-balance simulation compares policies on identical weather. Outputs are simulated quantities: irrigation depth, events, stress days and drainage. Validity depends on the assumptions in `scenario.yaml`. It is neither a prediction of field results nor a measured saving.

7.6 Why time series need chronological evaluation

Consecutive days of soil moisture are nearly copies of each other. If rows are shuffled before splitting, the training set contains days immediately before and after every test day, so the model can interpolate between its own neighbours. The score then reflects memorisation of the past and future around each test point, not the ability to forecast an unseen future. A chronological split mimics real use: train on the past, evaluate on a later period.

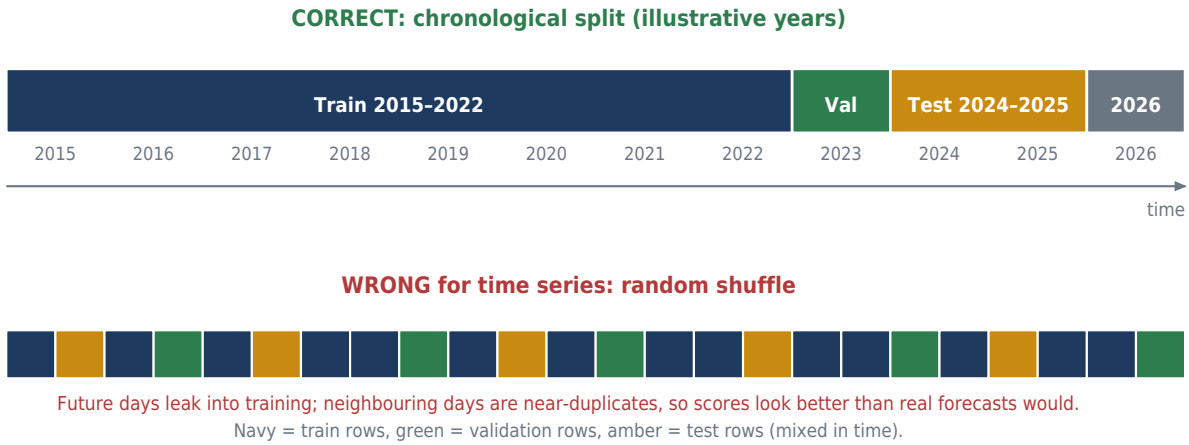


Figure 6. *Chronological versus shuffled splitting (years are illustrative).*

Synthetic or rule-generated labels

If any label is generated by the team's own rules (for example an 'irrigate' label created from a threshold), a model trained on it only learns the rule. Its accuracy says nothing about whether irrigation was actually needed. Such labels and the resulting scores must be marked **simulated** and never presented as verified real-world outcomes.

7.7 Evaluation formulas

For n test samples with true values y_i, predictions $\hat{y}_i$ and mean $\bar{y}$:

$$\text{MAE} = (1/n)\ \Sigma\ |y_i - \hat{y}_i|$$

$$\text{RMSE} = \sqrt{[\ (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2\ ]}$$

$$R^2 = 1 - \Sigma\ (y_i - \hat{y}_i)^2\ /\ \Sigma\ (y_i - \bar{y})^2$$

$$\text{Skill score} = 1 - \text{MAE}_{\text{model}}\ /\ \text{MAE}_{\text{persistence}}$$

MAE is the average absolute error in the original unit (m^3/m^3 for moisture). RMSE squares errors first, so it penalises large errors more. R^2 compares the model with always predicting the mean; on strongly autocorrelated series it can be high even for weak forecasts, so always compare with persistence.

For classification, with true positives TP, false positives FP, false negatives FN and true negatives TN:

$$\text{Accuracy} = (TP + TN)\ /\ (TP + TN + FP + FN)$$

$$\text{Precision} = TP\ /\ (TP + FP)\quad \text{Recall} = TP\ /\ (TP + FN)$$

$$F1 = 2 \cdot \text{Precision} \cdot \text{Recall}\ /\ (\text{Precision} + \text{Recall})\quad \text{Macro-F1} = \text{mean of per-class F1}$$

Percentage water difference in the simulation, where W is total irrigation applied over the same season and weather:

$$\text{Water difference (\%)} = (W_{\text{fixed}} - W_{\text{policy}})\ /\ W_{\text{fixed}} \times 100$$

Report this only together with stress days and drainage loss, and label it a *simulated estimate*. If W_{fixed} is zero the quantity is undefined.

8 Data Tables and Example Results

The tables below are templates and examples [I]. Values shown as "To be measured" must be replaced with real results generated by the team's scripts. No accuracy, error, yield or water-saving value is stated here because none exists yet.

Where each required table appears: dataset register (Table in Section 4), data dictionary template and sample schema (Section 4.5), weekly timeline (Section 5), model comparison, resource comparison and final deliverables (this section), testing plan (Section 10), team responsibility matrix (Section 11) and risk assessment (Section 12).

Table 31. Model comparison (placeholders)

Model	Task	Baseline	Test MAE	Test RMSE	Test R^2 / F1	Status
Persistence	Soil moisture t+1	—	To be measured	To be measured	To be measured	Not run
Random Forest	Soil moisture t+1	Persistence	To be measured	To be measured	To be measured	Not run
HistGradientBoosting (optional)	Soil moisture t+1	Persistence	To be measured	To be measured	To be measured	Not run
Random Forest	Crop classification	Majority class	n/a	n/a	Accuracy / macro-F1: To be measured	Not run
Previous-year / mean yield	Yield	—	To be measured	To be measured	To be measured	Not run
Random Forest or boosting	Yield	Previous-year yield	To be measured	To be measured	To be measured	Not run

Table 32. Resource comparison (simulated scenario; placeholders)

Policy	Irrigation applied (mm)	Events	Stress days	Drainage loss (mm)	Water difference vs fixed (%)
Fixed schedule (every N days)	To be simulated	To be simulated	To be simulated	To be simulated	— (reference)
Threshold only	To be simulated	To be simulated	To be simulated	To be simulated	To be simulated
Threshold + rain deferral + forecast	To be simulated	To be simulated	To be simulated	To be simulated	To be simulated

Both tables are templates. Any entry in them must be labelled "simulated" or "held-out test result" with the date and script that produced it.

Table 33. Final deliverables

#	Deliverable	Format	Week	Owner (suggested)
1	Source code repository (tagged v1.0)	Git	12	Prashant
2	Dataset register and licence notes	CSV/Markdown	2	Sachin
3	Data-quality report and EDA figures	Markdown/PNG	3	Sachin
4	Feature pipeline and leakage tests	Python/pytest	4	Sachin, Ravi
5	Soil-moisture model, metrics, plots	joblib/CSV/PNG	5	Ravi
6	Recommendation engine and rule documentation	Python/YAML	6	Ravi
7	Crop classifier, nutrient guidance	joblib/YAML	7	Kartik
8	Yield model and evaluation	joblib/CSV	8	Kartik
9	Scenario simulation and comparison table	Python/CSV	9	Ravi
10	Streamlit dashboard with real screenshots	Python/PNG	10	Prashant
11	Test report and reproducibility log	Markdown	11	Prashant, all
12	Final report, slides, demo video, viva notes	PDF/PPTX/MP4	12	All

9 Dashboard Design

The dashboard is a **proposed design** [A]. The figures below are wireframes, not screenshots of a built application. After Week 10 they should be replaced by real screenshots.



Figure 7. *Proposed wireframe of the Irrigation planner page (design only; curve and values are placeholders).*



Figure 8. *Proposed layout of all eight pages (design only).*

Table 34. Page specifications

Page	Charts and displays	Controls	Key metrics shown	User interactions
1 Overview	Status tiles, site map, data-mode indicator, latest recommendation summary	Location selector	Latest date, data mode, model version	Navigate to detail pages
2 Soil and weather	Time series of moisture, rainfall, temperature, ET0; seasonal view	Date range, variable and layer selectors	Missing-data %, units, provenance badge	Zoom, download CSV
3 Irrigation planner	Moisture history and forecast with crop trigger line; recommendation card	Location, crop, horizon, soil parameters, optional system parameters	Forecast moisture, relative saturation, rain outlook, action	Change crop or thresholds; see reason update; download recommendation
4 Crop suitability	Top-3 bar chart with probabilities	Inputs for N, P, K, pH, temperature, humidity, rainfall (dataset units)	Predicted crops, probabilities	Enter values; compare with typical ranges
5 Fertilizer guidance	Nutrient status table (low/adequate/high), pH range chart	Crop, soil-test form (lab units)	Status per nutrient, rule source	Submit form; read advice; download summary
6 Yield prediction	Predicted yield against district history	Crop, district, season, year	Predicted yield, historical range, model error from evaluation	Switch crop/district; read limitation note
7 Model evaluation	Actual vs predicted, confusion matrix, baseline comparison tables	Model selector, test period	MAE, RMSE, R^2; accuracy, macro-F1	Switch models; download tables
8 Scenario simulator	Soil-water curves and bar charts per policy	Scenario parameters, policy choice, seed	Water applied, events, stress days (simulated)	Edit assumptions; re-run; download table

Design rules

Every chart shows units and a provenance badge. Every recommendation shows its reason. Every simulated result carries the word "simulated". The sidebar shows online/offline mode and the Open-Meteo attribution.

10 Testing and Evaluation Plan

Three different questions are answered separately [A]: **(1) How accurate are the models?** (held-out metrics); **(2) Does the software behave correctly?** (application tests); **(3) Does it help in a real field?** (not measurable here; future work).

10.1 Model assessment with held-out data

- Each dataset is split before any modelling. The test set is used once, after all choices are made.
- Soil moisture: chronological test period never used for tuning. Yield: later years (and held-out districts). Crop classification: stratified held-out 20 %.
- Every model is compared with a baseline; improvements are reported with the same metric on the same rows.
- Report uncertainty: cross-validation spread, error by season and by condition (rain vs no rain).
- State all limitations next to the results, including provenance of the target variable.

Table 35. Testing matrix

ID	Area	Test	Method	Expected result	Week
T01	Missing columns	Load file without a required column	pytest with a modified fixture	Clear error naming the column	3
T02	Invalid timestamps	Unparseable or out-of-order dates	pytest	Rejected or sorted with a warning	3
T03	Missing values	Gaps in moisture/rain	pytest and EDA	Handled by documented rule; counts logged	3
T04	Duplicates	Duplicate dates or rows	pytest	Detected; duplicates removed or error raised	3
T05	Unit validation	Moisture 0–100 instead of 0–1; negative rain	pytest	Flagged before modelling	3
T06	Data leakage	Target shift, rolling window, split order	pytest on toy series	No future information in features	4
T07	Split validity	Train < validation < test dates; stratification	pytest	Order holds; class proportions equal	4, 7
T08	Model loading	Load saved model in a new process	script	Same predictions on stored sample	5, 11
T09	Prediction errors	Wrong column order, NaN input, unseen category	pytest	Meaningful error or graceful default	5, 8
T10	Recommendation logic	Wet, dry, rain-expected, missing-data cases	pytest parametrised	NO_IRRIGATION, IRRIGATE, DEFER, NO_RECOMMENDATION	6
T11	Simulation	Water balance in a hand-calculated toy case	pytest	Mass balance holds; bounds respected	9
T12	API failure	requests raises ConnectionError or timeout	monkeypatch	Fallback to cache; offline badge shown	10, 11
T13	Offline operation	Run app with the network disabled	manual and smoke test	All pages work with local data	10, 11
T14	Dashboard behaviour	Valid and invalid inputs on each page	manual checklist	No tracebacks; helpful messages	10, 11
T15	Downloads	CSV export	manual	File opens; columns as documented	10
T16	Reproducibility	Two runs from a clean environment	script comparing metrics and hashes	Identical metrics with the same seed	11
T17	Provenance labels	Check badges and 'simulated' labels	manual review	Every modelled/simulated value labelled	10, 12

10.2 Three levels of evaluation

Table 36. Model metrics, application tests and real-world impact are different

Level	Question	Evidence	Available in this project?
Model metrics	How close are predictions to held-out data?	MAE, RMSE, R^2, accuracy, F1	Yes, after Weeks 5–8
Application testing	Does the code do what it should?	pytest results, manual UI checks	Yes, Week 11
Real-world agricultural impact	Does water use or yield change on farms?	Field trial with measured water and yield	No (future work)

11 Team Responsibilities

The team consists of the four members listed in the Phase-I report [R]. The assignments below are **suggestions** [A] that the team can modify. Each member should still be able to explain every module in the viva.

Table 37. Responsibility matrix (L = lead, S = support, R = reviewer)

Area	Sachin Raj	Ravi Raj Kumar	Kartik Sharma	Prashant Kumar Yadav
Dataset acquisition, register, preprocessing, EDA	L	R	S	S
Feature engineering and ML pipeline	L	S	R	S
Soil-moisture forecasting	S	L	R	S
Irrigation recommendation engine	R	L	S	S
Crop suitability and fertiliser guidance	S	R	L	S
Crop-yield prediction	S	R	L	S
Resource optimization (simulation)	S	L	R	S
Dashboard integration	S	S	S	L
Testing and reproducibility	S	S	S	L
Documentation, report, presentation	S	S	S	L

Table 38. Mapping of members to weekly deliverables (suggested)

Member	Primary focus	Weeks	Key deliverables
Sachin Raj (Roll 2410030656)	Dataset acquisition and preprocessing	2, 3, 4	Raw data and manifest, dataset register, data-quality report, features.py, leakage tests
Ravi Raj Kumar (Roll 2410030680)	Soil-moisture forecasting and irrigation	5, 6, 9	Moisture model and metrics, recommendation engine, simulator and comparison table
Kartik Sharma (Roll 2410030213)	Crop suitability, fertiliser guidance and yield	7, 8	Crop classifier, nutrient rules, yield model and evaluation
Prashant Kumar Yadav (Roll 2410030236)	Dashboard, integration, testing, documentation	10, 11, 12 (and repository from week 1)	Streamlit app, test suite, reproducibility log, final report and slides
All members	Planning and final delivery	1, 12	Scope, architecture, rehearsals, viva preparation

Working agreements

Weekly 30-minute stand-up; weekly review with the guide. Short-lived feature branches, pull requests reviewed by a different member, tests required before merge. Each member updates their rows in the dataset register and the known-issues log.

12 Risk Assessment

Likelihood and impact are team-level qualitative judgements [I] that should be revisited at each weekly review.

Table 39. Risk register

#	Risk	Impact	Likelihood	Mitigation	Fallback
1	No physical sensors	High: no direct measurements	Certain	Use reanalysis, satellite and public data; label provenance; design a sensor-ready interface.	Present results as modelled; describe IoT as future work.
2	Incomplete soil-moisture data	Medium	Medium	Use gap-aware aggregation; choose a location with full reanalysis coverage; optional SMAP check.	Shorter period or a different location.
3	Incompatible datasets (different units, locations, years)	High	High	Keep each dataset in its own model; join only where keys exist; document assumptions.	Separate models without cross-dataset joins.
4	Missing fertiliser dose labels	Medium	High	Rule-based guidance with cited sources.	Display nutrient status only.
5	Limited or old yield data	Medium	Medium	Use ICRISAT/Kaggle district data; state limits.	Model without weather join; trend baseline.
6	API outage or rate limits	Medium	Medium	Cache every response; throttle requests; offline mode.	Bundled sample data.
7	Poor model performance (no gain over baseline)	Medium	Medium	Feature review, gradient boosting, error analysis.	Report honestly; keep the baseline in the app.
8	Data leakage producing misleading scores	High	Medium	Chronological splits, leakage tests, peer review.	Re-run after fixing; document the correction.
9	Insufficient implementation time	High	Medium	MoSCoW; start dashboard skeleton early; weekly checkpoints.	Drop optional items (SMAP, boosting, multi-horizon).
10	Team member unavailable	Medium	Low–Medium	Pair on each module; shared repo and documentation.	Reassign using the responsibility matrix.
11	Licence or attribution error	Medium	Low	Register records licence; attribution in app and report.	Remove or replace the dataset.

13 Presentation Preparation

A suggested 15-slide deck [A]. For every slide the status line says whether the content is planned or completed; update it as work progresses and never show planned work as completed.

Slide 1: Project title

Main content	Title, team, guide, institution, session, project code.
Suggested visual	Logo and title graphic.
Presenter notes	Introduce the project and team roles.
Points for faculty	State that this is Phase-I and a roadmap.
Status	Completed [A]

Slide 2: Problem statement

Main content	Fixed schedules and rule-of-thumb farming lead to wasted water, excess inputs and reactive management [R].
Suggested visual	Simple infographic of the three problems.
Presenter notes	Explain why timing matters for water and fertiliser.
Points for faculty	Quote the problem from the report faithfully.
Status	Completed (from report) [A]

Slide 3: Objectives and research gap

Main content	Five objectives; gap: single-component tools or costly systems; need a unified, predictive plan [R].
Suggested visual	Objectives table (Table in Section 2).
Presenter notes	Show how each objective maps to a module.
Points for faculty	Make clear which parts are adapted for no hardware.
Status	Completed (from report) [A]

Slide 4: Data acquisition without sensors

Main content	Open-Meteo reanalysis and forecast, Kaggle, ICRISAT, optional SMAP, user input.
Suggested visual	Provenance classes table; architecture left side.
Presenter notes	Explain modelled versus measured.
Points for faculty	Emphasise no hardware dependency and labelling of data.
Status	Planned, then Completed after Week 2 [A]

Slide 5: Preprocessing and EDA

Main content	Validation, daily aggregation, missing and duplicate handling, charts.
Suggested visual	Real EDA charts and the data-quality table.
Presenter notes	Walk through the checks.
Points for faculty	Show actual counts of removed rows.
Status	Planned [A]

Slide 6: Soil-moisture forecasting

Main content	Next-day moisture; persistence baseline; Random Forest; MAE, RMSE, R^2.
Suggested visual	Actual vs predicted chart; metrics table.
Presenter notes	Explain chronological testing.
Points for faculty	Report real numbers; say if baseline is hard to beat.
Status	Planned [A]

Slide 7: Irrigation recommendation engine

Main content	Rules on relative saturation, rain deferral, explanations.
Suggested visual	Decision flowchart; example recommendation.
Presenter notes	Show wet, dry and missing-data cases.
Points for faculty	Why duration needs system parameters.
Status	Planned **[A]**

Slide 8: Crop suitability and fertiliser guidance

Main content	Classifier with accuracy/F1; separate rule-based nutrient guidance.
Suggested visual	Confusion matrix; nutrient status table.
Presenter notes	Explain classification versus dose prediction.
Points for faculty	Dataset limits and units.
Status	Planned **[A]**

Slide 9: Crop-yield prediction

Main content	District yield from actual statistics; baselines; temporal test.
Suggested visual	Predicted vs actual chart; baseline table.
Presenter notes	Explain yield = production / area.
Points for faculty	District vs farm level; data period.
Status	Planned **[A]**

Slide 10: Resource optimization

Main content	Fixed schedule vs prediction-informed policy in simulation.
Suggested visual	Soil-water curves; comparison table.
Presenter notes	Explain assumptions and stress days.
Points for faculty	Simulated scenario, not field savings.
Status	Planned **[A]**

Slide 11: Dashboard design

Main content	Eight pages; offline mode; provenance badges.
Suggested visual	Wireframes now; real screenshots later.
Presenter notes	Walk through a user journey.
Points for faculty	Label the wireframes as design until built.
Status	Design completed; build planned **[A]**

Slide 12: Testing and evaluation

Main content	Held-out metrics, pytest matrix, reproducibility.
Suggested visual	Testing matrix; pytest output.
Presenter notes	Separate model metrics, application tests and real-world impact.
Points for faculty	Show pass counts only after running tests.
Status	Planned **[A]**

Slide 13: Twelve-week roadmap

Main content	Timeline with owners and deliverables.
Suggested visual	Gantt chart (Section 5).
Presenter notes	Show current week and next milestone.
Points for faculty	Mark finished weeks honestly.
Status	Completed (plan) **[A]**

Slide 14: Team responsibilities and risks

Main content	Responsibility matrix and top risks with mitigations.
Suggested visual	Matrix and risk table.
Presenter notes	Explain fallbacks.
Points for faculty	Say that roles are flexible.
Status	Completed (plan) **[A]**

Slide 15: Expected outcomes and future work

Main content	Deliverables; limits; IoT sensors, imagery/NDVI, field trials, recent yield data.
Suggested visual	Roadmap arrow to future extensions.
Presenter notes	Close with what was built and learned.
Points for faculty	Do not claim impact beyond the evidence.
Status	Completed (plan); update at the end **[A]**

14 Viva Preparation

Concise answers suitable for a B.Tech CSE viva [A]. Replace statements about results with the team's real findings after the experiments.

Q1. Why is this project needed?

Farms often irrigate and fertilise by schedule or habit, wasting water and inputs and reacting late to stress. Data-driven forecasts and recommendations can make timing more precise [R].

Q2. Why is machine learning appropriate?

Soil water depends non-linearly on rainfall, temperature, evaporation and season. ML learns such relationships from historical data and can forecast and classify faster than manual rules, while rules keep decisions explainable.

Q3. How do you obtain data without sensors?

From public sources: Open-Meteo reanalysis and forecasts, Kaggle and ICRISAT files, optional NASA SMAP, and user-entered soil tests. Each source is labelled by provenance.

Q4. What does soil moisture mean?

Usually volumetric water content: the volume of water per volume of soil (m^3/m^3). It differs from gravimetric moisture (mass-based) and soil suction.

Q5. What are reanalysis data?

Gridded estimates produced by running a numerical model constrained by assimilated observations over past years. ERA5-Land re-runs the land component at higher resolution (about 0.1°). The values are modelled, not sensor readings.

Q6. Regression versus classification?

Regression predicts a continuous value (moisture, yield). Classification predicts a category (crop). We use MAE/RMSE/R^2 for the first and accuracy/F1 for the second.

Q7. Why Random Forest?

It captures non-linear interactions, needs little feature scaling, is robust with moderate data and gives feature importances. It is a strong, explainable baseline; its weakness is that it cannot extrapolate beyond training values.

Q8. What do MAE and RMSE measure?

Both are error sizes in the target's unit. MAE averages absolute errors; RMSE squares them first, so large errors count more.

Q9. What is R^2 and why can it mislead?

R^2 is the share of variance explained relative to predicting the mean. On slowly changing series it can be high even for weak forecasts, so we also compare with persistence.

Q10. Why does time-series splitting matter?

Training must use the past and testing the future, as in real use. Shuffling mixes neighbouring days, so the test is not independent and scores look unrealistically good.

Q11. What is data leakage?

Using information in training or features that would not be available at prediction time, for example future rainfall or scalers fitted on all data. It inflates scores.

Q12. What is a persistence baseline?

Predicting that tomorrow equals today. Any useful model must beat it.

Q13. How does the recommendation engine work?

It computes relative saturation from forecast moisture, checks the rain outlook, compares with a crop-specific trigger and returns IRRIGATE, DEFER, NO_IRRIGATION or NO_RECOMMENDATION with a reason.

Q14. Why can't you say how many minutes to irrigate?

Duration needs the water depth required, root depth, field area, flow rate and system efficiency. Moisture alone does not give these; we compute it only when the user provides them.

Q15. Why is crop recommendation different from fertiliser prediction?

Crop recommendation chooses a crop label from conditions. Fertiliser dose is a numeric amount depending on crop, soil test, yield target and local norms; public data lack dose labels, so we use documented rules.

Q16. How is crop yield predicted?

Yield = production/area from district statistics is the target. Inputs are crop, season, region, year and optional weather aggregates. We test on later years against baselines.

Q17. How are water savings calculated?

In simulation: $(W_{fixed} - W_{policy})/W_{fixed} \times 100$, reported with stress days under stated assumptions. It is a simulated estimate, not a measured field saving.

Q18. How could IoT sensors be added later?

A sensor node (for example a capacitive moisture sensor on a microcontroller) can send readings to the same CSV or API interface. After calibration the readings can replace or refine the reanalysis input and retrain the model **[R]**.

Q19. What are the limitations of public agricultural datasets?

Coarse spatial level (district), old or incomplete years, unit inconsistencies, compiled or augmented data, missing soil and management information, and unclear licences.

Q20. How do you know your results are reproducible?

Fixed seeds, pinned versions, scripts that generate tables, saved model metadata and a clean-environment re-run.

Q21. What did the report say about the gap?

Existing work addresses single components or costs too much or is view-only; farmers need a unified predictive plan for water, fertiliser and labour **[R]**.

Q22. What is not implemented?

Imagery/NDVI analysis, pesticide targeting, labour optimization and field trials are outside this phase and listed as future work.

Q23. Why not claim real savings?

Without field measurements of water use and yield, any saving would be an assumption. We present simulated scenarios with explicit assumptions.

Q24. How do you validate the classifier?

Stratified held-out test and cross-validation with macro-F1 and a confusion matrix; we acknowledge that high scores on a compiled dataset may not transfer to real fields.

15 References and Research Integrity

URLs were located through web searches on 28 October 2026. "Seen in results" means the address (or documentation for it) appeared in retrieved sources; "Open to confirm" means the standard documentation address was not opened in this session. The team must open each link and confirm the description before final submission.

Table 40. Sources

#	Source	Use and accurate description	URL	Check
1	Open-Meteo Historical Weather API	Reanalysis-based hourly history; ERA5 (0.25°) and ERA5-Land (0.1°); soil moisture layers; /v1/archive endpoint.	https://open-meteo.com/en/docs/historical-weather-api	Seen in results
2	Open-Meteo Weather Forecast API	Hourly and daily forecasts; past_days and forecast_days parameters; soil moisture variables.	https://open-meteo.com/en/docs	Seen in results
3	Open-Meteo terms	Licence CC BY 4.0, free non-commercial use, attribution.	https://open-meteo.com/en/features#terms	Seen as terms link in API spec
4	Open-Meteo Geocoding API	Location search to coordinates (for district centres).	https://open-meteo.com/en/docs/geocoding-api	Seen in results
5	Kaggle API (official repository)	Command-line download of datasets (kaggle datasets download -d owner/dataset).	https://github.com/Kaggle/kaggle-api	Open to confirm (forks seen)
6	Kaggle documentation	Account, API token and dataset usage.	https://www.kaggle.com/docs/api	Open to confirm
7	Kaggle: Crop Recommendation Dataset	2,200 rows, 22 crops, N, P, K, temperature, humidity, pH, rainfall.	https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset	Seen as cited in papers
8	Kaggle: Crop Production in India	District-level area and production by crop and season (reported 1997–2015).	https://www.kaggle.com/datasets/abhinand05/crop-production-in-india	Seen as cited in analyses
9	ICRISAT District Level Database	District-wise yearly area, production and yield for major crops, open access with citation.	http://data.icrisat.org/dld/index.html	Seen in results
10	ICRISAT DLD crop files	Crop area, production and yield data files.	http://data.icrisat.org/dld/src/crops.html	Seen in results
11	ICRISAT DLD citation	Dataset DOI.	https://doi.org/10.21421/D2/XFB1BZ	Seen in results
12	NASA/NSIDC SMAP L3 36 km soil moisture (SPL3SMP)	Daily surface soil moisture on 36 km EASE-Grid 2.0; Earthdata Login required.	https://nsidc.org/data/spl3smp/	Seen in results
13	SMAP SPL3SMP DOI	Citation DOI for Version 9.	https://doi.org/10.5067/4XXOGX0OOW1S	Seen in results
14	scikit-learn documentation	RandomForestRegressor, TimeSeriesSplit, metrics, pipelines.	https://scikit-learn.org/stable/	Seen in results
15	scikit-learn: TimeSeriesSplit	Time-ordered cross-validation (successive training sets are supersets).	https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html	Pattern seen (older versions); open to confirm
16	Streamlit documentation	App framework; st.download_button, caching.	https://docs.streamlit.io	Seen in results
17	Streamlit: st.download_button	Download widget.	https://docs.streamlit.io/develop/api-reference/widgets/st.download_button	Seen in results
18	pandas / Plotly / joblib / pytest	Standard library documentation.	https://pandas.pydata.org/docs/ https://plotly.com/python/ https://joblib.readthedocs.io https://docs.pytest.org	Open to confirm

Phase-I report references (as listed in the report) [R]

The report lists six IEEE publications by title only, without authors, years or DOIs. They are reproduced below exactly as titled; the team should add full bibliographic details from the publisher's pages before

final submission. No additional papers have been invented for this document.

1. "Machine Learning Based Soil Moisture Prediction for Internet of Things Based Smart Irrigation System," IEEE Conference Publication, 2019.
2. "Predictive Water Management for Precision Farming Using IoT & Bi-LSTM Networks," IEEE Conference Publication.
3. "Development of an IoT Based Data Acquisition and Automatic Irrigation System for Precision Agriculture," IEEE Conference Publication.
4. "Construction of an Intelligent Analysis System for Crop Health Status Based on Drone Remote Sensing Data and CNN," IEEE Journals & Magazine.
5. "Applying Machine Learning Approaches in Fertilizer Optimization and Yield Prediction for Smart Agriculture," IEEE Conference Publication.
6. "Smart Agriculture Monitoring System Using Internet of Things (IoT)," IEEE Conference Publication.

Separating kinds of information

Table 41. How information is classified in this document

Kind	Where it appears	Marker
Phase-I report content	Problem, objectives, scope, gap, team, feasibility remarks	[R]
Implementation recommendations	Weekly plan, architecture, methodology, dashboard, tests, team matrix, risks, presentation	[A]
Externally verified technical information	Open-Meteo, SMAP, Kaggle and ICRISAT facts cited above	[V]
Illustrative examples and placeholders	Split dates, example thresholds, simulated scenarios, empty result tables	[I]
Actual experimental results	None yet; to be inserted from the team's real runs	[E]

Research-integrity rules for the team

Never present modelled or simulated values as measurements. Never state accuracy, error, yield or water-saving figures that were not produced by your own code. Cite every dataset and API. Re-check licences and URLs before submission. Report negative results (for example, a model that does not beat persistence) honestly.

16 Final Implementation and Presentation Checklist

Implementation

- ☐ Phase-I objectives traced to modules and weeks
- ☐ Pilot location and crops fixed and documented
- ☐ All datasets downloaded; raw files untouched; manifest with hashes
- ☐ Dataset register complete (source, licence, provenance, limitations)
- ☐ Data-quality report written; processed tables saved
- ☐ Features and targets built; chronological splits fixed; leakage tests pass
- ☐ Soil-moisture model trained; compared with persistence; metrics on the test set only
- ☐ Recommendation engine implemented; wet, dry and missing-data tests pass
- ☐ Crop classifier evaluated; nutrient rules sourced
- ☐ Yield model evaluated on held-out years and districts
- ☐ Resource simulation run; assumptions documented; results labelled simulated
- ☐ Dashboard built (8 pages); offline mode verified; downloads work
- ☐ pytest suite passes on a clean environment; reproducibility verified
- ☐ Attribution to Open-Meteo shown in the app and report
- ☐ README verified by a fresh clone; release tagged

Presentation and viva

- ☐ Slides use only real screenshots and real numbers; planned items are labelled
- ☐ Every number in slides traces to a file in reports/
- ☐ Wireframes replaced by real screenshots (or still labelled as design)
- ☐ Presenters assigned; two timed rehearsals completed
- ☐ Offline demo and recorded backup video ready
- ☐ Limitations slide prepared (modelled data, district level, simulation)
- ☐ Viva questions practised by every member on every module
- ☐ References opened and verified; Phase-I report references completed
- ☐ Report inconsistencies (section label, school name) confirmed with the guide

Closing note

This roadmap is a plan. Its value will be judged by how honestly the team reports what it builds and measures. Fill every "To be measured" cell only with results your own code produces.